

TITRE PROFESSIONNEL  
CONCEPTEUR DÉVELOPPEUR D'APPLICATIONS  
Niveau 6 (Cadre national des certifications)  
RNCP 37873 — Code titre TP-01281

---

# Dossier de projet

## Solage

*Réseau social de microblogging — application web sécurisée multicouche*

---

**Candidat :**

Adam COURTARO

**Session :**

**Projet :**

Réalisé en formation

**Centre :**

Dossier rendant compte de la mise en œuvre des onze compétences professionnelles du titre, réparties sur les trois certificats de compétences professionnelles (CCP).

*Document de 40 à 60 pages hors page de garde, sommaire et annexes — schémas et illustrations compris.*

# Table des matières

|          |                                                          |          |
|----------|----------------------------------------------------------|----------|
| <b>1</b> | <b>Liste des compétences mises en œuvre</b>              | <b>1</b> |
| <b>2</b> | <b>Expression des besoins</b>                            | <b>3</b> |
| 2.1      | Contexte et problème adressé . . . . .                   | 3        |
| 2.2      | Objectifs du projet . . . . .                            | 3        |
| 2.3      | Périmètre et limites . . . . .                           | 4        |
| 2.4      | Acteurs et cas d’usage . . . . .                         | 4        |
| 2.5      | Contraintes réglementaires et qualité . . . . .          | 4        |
| <b>3</b> | <b>Environnement technique</b>                           | <b>6</b> |
| 3.1      | Vue d’ensemble de la pile technique . . . . .            | 6        |
| 3.2      | Justification des choix techniques . . . . .             | 6        |
| 3.3      | Poste de travail et outils . . . . .                     | 7        |
| 3.4      | Conteneurisation (Docker) . . . . .                      | 7        |
| <b>4</b> | <b>Réalisations</b>                                      | <b>9</b> |
| 4.1      | Architecture logicielle multicouche . . . . .            | 9        |
| 4.1.1    | Règles de séparation des couches . . . . .               | 9        |
| 4.1.2    | Rôle de sécurité de chaque couche (DICP) . . . . .       | 9        |
| 4.1.3    | Patrons mis en œuvre . . . . .                           | 10       |
| 4.2      | Gestion de projet . . . . .                              | 11       |
| 4.3      | Maquettes et enchaînement des écrans . . . . .           | 12       |
| 4.4      | Conception de la base de données . . . . .               | 14       |
| 4.4.1    | Modèle conceptuel des données (MCD) . . . . .            | 14       |
| 4.4.2    | Modèle logique / physique (MLD, MPD) . . . . .           | 14       |
| 4.4.3    | Évolution du schéma : migrations idempotentes . . . . .  | 15       |
| 4.5      | Diagrammes UML . . . . .                                 | 18       |
| 4.5.1    | Diagramme de cas d’utilisation . . . . .                 | 18       |
| 4.5.2    | Diagramme de séquence : « Publier un message » . . . . . | 18       |
| 4.5.3    | Diagramme de classes (couche modèle) . . . . .           | 19       |
| 4.6      | Composants d’interface utilisateur . . . . .             | 20       |
| 4.7      | Composants métier . . . . .                              | 22       |
| 4.7.1    | Contrôle d’accès : la parade à l’IDOR . . . . .          | 22       |
| 4.7.2    | Authentification <i>vs</i> autorisation . . . . .        | 23       |

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| 4.7.3    | Validation pure, découplée de la sortie HTTP . . . . .            | 23        |
| 4.8      | Composants d'accès aux données . . . . .                          | 24        |
| 4.9      | Autres composants : le socle « framework » . . . . .              | 25        |
| 4.10     | Sécurité de l'application . . . . .                               | 26        |
| 4.10.1   | XSS : échappement à la sortie . . . . .                           | 27        |
| 4.10.2   | CSRF : jeton de synchronisation . . . . .                         | 27        |
| 4.10.3   | En-têtes de sécurité et cookie de session . . . . .               | 28        |
| 4.10.4   | IDOR et contrôle d'accès . . . . .                                | 28        |
| 4.10.5   | Axes d'amélioration identifiés . . . . .                          | 28        |
| 4.11     | Plan de tests . . . . .                                           | 29        |
| 4.12     | Jeu d'essai de la fonctionnalité la plus représentative . . . . . | 30        |
| 4.13     | Préparation et documentation du déploiement . . . . .             | 31        |
| 4.14     | Contribution à la mise en production (DevOps) . . . . .           | 33        |
| 4.15     | Veille de sécurité . . . . .                                      | 35        |
| <b>5</b> | <b>Bilan, difficultés et perspectives</b>                         | <b>37</b> |
| 5.1      | Satisfactions . . . . .                                           | 37        |
| 5.2      | Difficultés rencontrées et résolues . . . . .                     | 37        |
| 5.3      | Perspectives . . . . .                                            | 37        |
| <b>A</b> | <b>Annexes</b>                                                    | <b>38</b> |
| A.1      | Gestion de projet : suivi des tâches . . . . .                    | 38        |
| A.2      | Script de création de la base de données . . . . .                | 38        |
| A.3      | Code d'un autre composant : le routeur . . . . .                  | 39        |
| A.4      | Jeux de tests complets . . . . .                                  | 40        |
| A.5      | Maquettes et captures d'écran . . . . .                           | 43        |
| A.5.1    | Écrans bureau . . . . .                                           | 43        |
| A.5.2    | Écrans mobiles . . . . .                                          | 44        |

# Chapitre 1

## Liste des compétences mises en œuvre

Ce dossier rend compte de la réalisation du projet SOLAGE, une application web de microblogging développée en formation, support de la mise en œuvre des **onze compétences professionnelles** du titre *Concepteur Développeur d'Applications* (RNCP 37873), réparties sur les trois certificats de compétences professionnelles (CCP).

Le tableau 1.1 constitue la *grille de lecture* du jury : chaque compétence y est reliée à la preuve concrète qui la démontre dans SOLAGE et au chapitre où elle est développée.

**Compétences transversales.** Elles sont évaluées à *travers* les compétences professionnelles :

- **Communiquer en français et en anglais** : dossier et présentation structurés en français ; lecture de documentation technique en anglais (OWASP, manuel PHP) et *nommage du code en anglais* — identifiants, méthodes, messages de *commit* (niveau B1) ;
- **Mettre en œuvre une démarche de résolution de problème** : voir le chapitre 5 (bugs diagnostiqués et résolus : *headers already sent*, requête N+1) ;
- **Apprendre en continu** : veille de sécurité ayant conduit à la correction d'une faille IDOR (section 4.15).

| #   | Compétence                                                     | CCP | Preuve principale dans Solage                                                                          | Statut |
|-----|----------------------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------|--------|
| C1  | Installer et configurer son environnement de travail           | 1   | Docker (FrankenPHP, Caddy, Traefik, PostgreSQL), Git, Composer, conteneurs dev = prod                  | OK     |
| C2  | Développer des interfaces utilisateur                          | 1   | Vues <code>modules/views/</code> , JS vanilla, AJAX, échappement XSS, CSP                              | OK     |
| C3  | Développer des composants métier                               | 1   | Contrôleurs <code>modules/controllers/</code> , autorisation, contrôle IDOR, CSRF                      | OK     |
| C4  | Contribuer à la gestion d'un projet informatique               | 1   | Git (commits conventionnels), <code>README.md</code> en tableau Kanban, journal de décisions technique | OK     |
| C5  | Analyser les besoins et maquette une application               | 2   | Expression des besoins (ch. 2), maquettes (Penpot) & enchaînement                                      | OK     |
| C6  | Définir l'architecture logicielle d'une application            | 2   | MVC multicouche, Router, middlewares, schéma DICP                                                      | OK     |
| C7  | Concevoir et mettre en place une base de données relationnelle | 2   | <code>solage.pg.sql</code> , migrations idempotentes, <code>seed.sql</code>                            | OK     |
| C8  | Développer des composants d'accès aux données SQL et NoSQL     | 2   | <code>modules/models/</code> , PDO requêtes préparées, anti-injection                                  | OK     |
| C9  | Préparer et exécuter les plans de tests                        | 3   | Plan de tests + tests unitaires / sécurité (PHPUnit)                                                   | OK     |
| C10 | Préparer et documenter le déploiement                          | 3   | <code>docker-compose.prod.yml</code> , Traefik HTTPS, <code>DEPLOYMENT.md</code>                       | OK     |
| C11 | Contribuer à la mise en production (DevOps)                    | 3   | Conteneurs, migrations, intégration continue (CI)                                                      | OK     |

**Table 1.1** — Cartographie des onze compétences professionnelles et de leur preuve dans SOLAGE.

## Chapitre 2

# Expression des besoins

SOLAGE étant un projet réalisé *pendant la formation*, sans commanditaire externe, c’est moi qui formule l’expression des besoins définissant les objectifs et les limites du projet, comme le prévoit le référentiel de certification.

### 2.1 Contexte et problème adressé

SOLAGE est un **réseau social de microblogging** inspiré de X (anciennement Twitter) : un utilisateur publie de courts messages, éventuellement accompagnés d’une image, auxquels les autres peuvent répondre dans un fil imbriqué et réagir par un « j’aime ».

Le besoin pédagogique sous-jacent est de se confronter à *l’ensemble des briques d’une application web sécurisée multicouche* sur un périmètre fonctionnel maîtrisable : authentification, opérations de création / lecture / mise à jour / suppression (CRUD), relations récursives (réponses imbriquées), téléversement de fichiers, recherche et modération. Ce périmètre, volontairement resserré, permet de traiter **la sécurité à chaque couche** plutôt que d’empiler des fonctionnalités superficielles.

### 2.2 Objectifs du projet

Les objectifs sont formulés de manière *vérifiable* :

- permettre à un visiteur de **s’inscrire**, de **se connecter** et de se déconnecter ;
- permettre à un utilisateur authentifié de **publier** un message (avec image facultative), d’y **répondre** dans un fil imbriqué, de **l’aimer**, de **rechercher** des utilisateurs et des messages, et d’**éditer son profil** ;
- permettre à un **administrateur** de **modérer** (supprimer tout contenu, accéder à une page d’administration) ;
- garantir, de manière transverse, la **sécurité applicative** (protection contre les failles XSS, CSRF, injection SQL, IDOR) conformément aux recommandations de l’OWASP et de l’ANSSI, ainsi que la conformité au RGPD et la prise en compte du RGAA.

## 2.3 Périmètre et limites

Le **hors-périmètre** est assumé explicitement : il traduit un arbitrage de maturité, et non un oubli. Ne font pas partie du projet :

- la messagerie privée entre utilisateurs ;
- les notifications en temps réel (WebSocket) ;
- la fédération inter-instances et les API publiques ouvertes ;
- toute fonctionnalité de paiement ou de monétisation.

## 2.4 Acteurs et cas d’usage

Trois acteurs interagissent avec le système, selon une relation de *généralisation* : l’administrateur est un utilisateur particulier, lui-même un visiteur authentifié.

| Acteur         | Cas d’usage principaux                                                                                  |
|----------------|---------------------------------------------------------------------------------------------------------|
| Visiteur       | Consulter la page de connexion, s’inscrire, se connecter                                                |
| Utilisateur    | Consulter le fil, publier, répondre, aimer, rechercher, éditer son profil, supprimer son propre contenu |
| Administrateur | Tous les cas de l’utilisateur <i>plus</i> la modération (supprimer tout contenu, page d’administration) |

**Table 2.1** — Acteurs et cas d’usage.

Quelques *user stories* représentatives, qui serviront de base au diagramme de cas d’utilisation (section 4.5) et au plan de tests (chapitre 4.11) :

- *En tant que* visiteur, *je veux* créer un compte *afin de* publier des messages.
- *En tant qu’*utilisateur, *je veux* publier un message avec une image *afin de* partager du contenu.
- *En tant qu’*utilisateur, *je veux* répondre à un message *afin de* participer à une conversation.
- *En tant qu’*administrateur, *je veux* supprimer n’importe quel message *afin de* modérer la plateforme.

## 2.5 Contraintes réglementaires et qualité

**RGPD.** L’application traite des données à caractère personnel (adresse email, mot de passe, contenus publiés). Le mot de passe n’est jamais stocké en clair : il est haché (`bcrypt`). La collecte est minimale (principe de minimisation). Des mentions légales et une information sur la finalité du traitement doivent accompagner la mise en production.

**RGAA (accessibilité).** Les interfaces visent un socle d’accessibilité : attributs `alt` sur les images, contrastes suffisants, libellés de formulaire, navigation au clavier et attributs `aria-*` sur les éléments interactifs (par exemple le champ de saisie de message porte `role="textinput"` et `aria-label`).

**Éco-conception.** Plusieurs choix limitent l’empreinte : minification des assets en production, pagination du fil (LIMIT 20), et suppression d’une requête N+1 réduisant le nombre d’allers-retours vers la base (section 4.8).



## Chapitre 3

# Environnement technique

### 3.1 Vue d'ensemble de la pile technique

SOLAGE repose sur une pile volontairement légère, dont je maîtrise chaque brique.

| Couche               | Technologie                                         | Rôle                                                                |
|----------------------|-----------------------------------------------------|---------------------------------------------------------------------|
| Langage              | PHP 8.3<br>( <code>declare(strict_types=1)</code> ) | Logique applicative                                                 |
| SGBD                 | PostgreSQL 16 via PDO                               | Persistance relationnelle                                           |
| Serveur applicatif   | FrankenPHP (Caddy intégré)                          | Exécution de PHP, service du statique                               |
| Reverse proxy / edge | Traefik v3                                          | Routage, terminaison TLS (Let's Encrypt)                            |
| Conteneurisation     | Docker, <code>docker compose</code>                 | Reproductibilité dev = prod                                         |
| Dépendances          | Composer                                            | <code>minify</code> , <code>phpdotenv</code> , <code>psr/log</code> |
| Front-end            | HTML, CSS, JavaScript vanilla                       | Interfaces et appels AJAX ( <code>fetch</code> )                    |
| Gestion de versions  | Git                                                 | Historique et traçabilité                                           |

Table 3.1 – Pile technique de SOLAGE.

### 3.2 Justification des choix techniques

Chaque choix est un compromis explicite, formulé selon le schéma « X plutôt que Y parce que Z, en acceptant le coût W ».

**MVC maison plutôt qu'un framework (Symfony, Laravel).** Objectif pédagogique : *comprendre et posséder* le routeur, l'autoloader et le cycle requête / réponse, plutôt que de déléguer à de la « magie » non maîtrisée. Coût assumé : réécrire des briques qu'un framework fournirait. En contexte professionnel, un framework serait retenu.

**PostgreSQL plutôt que MariaDB.** Typage strict, séquences (`SERIAL`), robustesse et conformité SQL. Le projet a d'ailleurs été *migré* de MariaDB vers PostgreSQL, ce qui a imposé d'adapter le schéma (`AUTO_INCREMENT` → `SERIAL`, `datetime` → `TIMESTAMP`).

**FrankenPHP plutôt qu'Apache ou Nginx + PHP-FPM.** Un seul binaire embarquant Caddy, terminaison TLS automatique, image Docker simple.

**Traefik plutôt que Nginx en reverse proxy.** Configuration *par labels Docker*, certificats Let's Encrypt automatiques, découverte dynamique des services.

**PDO plutôt qu'une extension native.** Abstraction portable et, surtout, **requêtes préparées** qui neutralisent l'injection SQL (section 4.8).

### 3.3 Poste de travail et outils

L'environnement de développement réunit : un IDE, **Git** pour la gestion de versions (historique de *commits* conventionnels, une fonctionnalité par série de commits), **Composer** pour les dépendances (`composer.json` / `composer.lock`), et une gestion rigoureuse des **secrets**. Les identifiants de base de données ne figurent jamais dans le code : ils sont chargés depuis un fichier `.env` (ignoré par Git) au moyen de la bibliothèque `vlucas/phpdotenv`, un gabarit `.env.example` documentant les variables attendues.

```

1 $envPath = dirname(__DIR__);
2 if (file_exists($envPath . '/.env')) {
3     $dotenv = Dotenv\Dotenv::createImmutable($envPath);
4     $dotenv->load();
5 }
6 // ... lecture de DB_HOST, DB_NAME, DB_USER, DB_PASSWORD ...
7 $dsn = "pgsql:host={$host};port={$port};dbname={$dbname};options='--
      client_encoding=UTF8'";
8 $options = [
9     PDO::ATTR_ERRMODE          => PDO::ERRMODE_EXCEPTION,
10    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
11    PDO::ATTR_EMULATE_PREPARES  => false,    // vraies requêtes préparées côté
      serveur
12 ];
13 $this->connection = new PDO($dsn, $user, $password, $options);

```

**Listing 3.1** — Chargement des secrets et connexion PDO — `includes/database.php`

La désactivation de `ATTR_EMULATE_PREPARES` force de *vraies* requêtes préparées côté serveur PostgreSQL, renforçant la protection contre l'injection SQL.

### 3.4 Conteneurisation (Docker)

L'application est décrite par deux piles `docker compose` : l'une pour le développement, l'autre pour la production.

**Développement.** Traefik en HTTP, FrankenPHP en *bind-mount* du code (rechargement à chaud), PostgreSQL exposé sur le port 5432 (pour les outils locaux), et un service `migrate` exécuté une seule fois avant le démarrage de l'application. Un *healthcheck* PostgreSQL garantit l'ordre de démarrage.

```

1 app:
2   build: { context: ., dockerfile: Dockerfile }
3   image: solage-app

```

```
4     volumes:
5         - ./:/app          # rechargement du code à chaud en dev
6         - /app/vendor      # vendor/ provient de l'image
7     depends_on:
8         postgres: { condition: service_healthy }
9         migrate:  { condition: service_completed_successfully }
10    labels:
11        - traefik.enable=true
12        - traefik.http.routers.solage.rule=Host('localhost')
13
14    postgres:
15        image: postgres:16-alpine
16        healthcheck:
17            test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
18            interval: 5s
```

**Listing 3.2** – Extrait du `docker-compose.yml` (développement) : service applicatif et base

**Production.** Traefik en HTTPS (Let's Encrypt, redirection HTTP → HTTPS), en-tête HSTS, FrankenPHP servi depuis l'image (sans *bind-mount*), et PostgreSQL *non exposé* (accessible uniquement sur le réseau interne). Le détail figure en section 4.13 (préparation du déploiement).

# Chapitre 4

## Réalisations

Ce chapitre, cœur du dossier, présente les réalisations qui mettent en œuvre les compétences : l’architecture logicielle, la gestion de projet, la conception (maquettes, modèle de données, diagrammes UML), le développement (interfaces, composants métier, accès aux données), la sécurité, puis les tests et la veille.

### 4.1 Architecture logicielle multicouche

SOLAGE suit une architecture **MVC multicouche**, où chaque couche porte une responsabilité unique *et* un rôle de sécurité. La figure 4.1 en donne la vue d’ensemble.

#### 4.1.1 Règles de séparation des couches

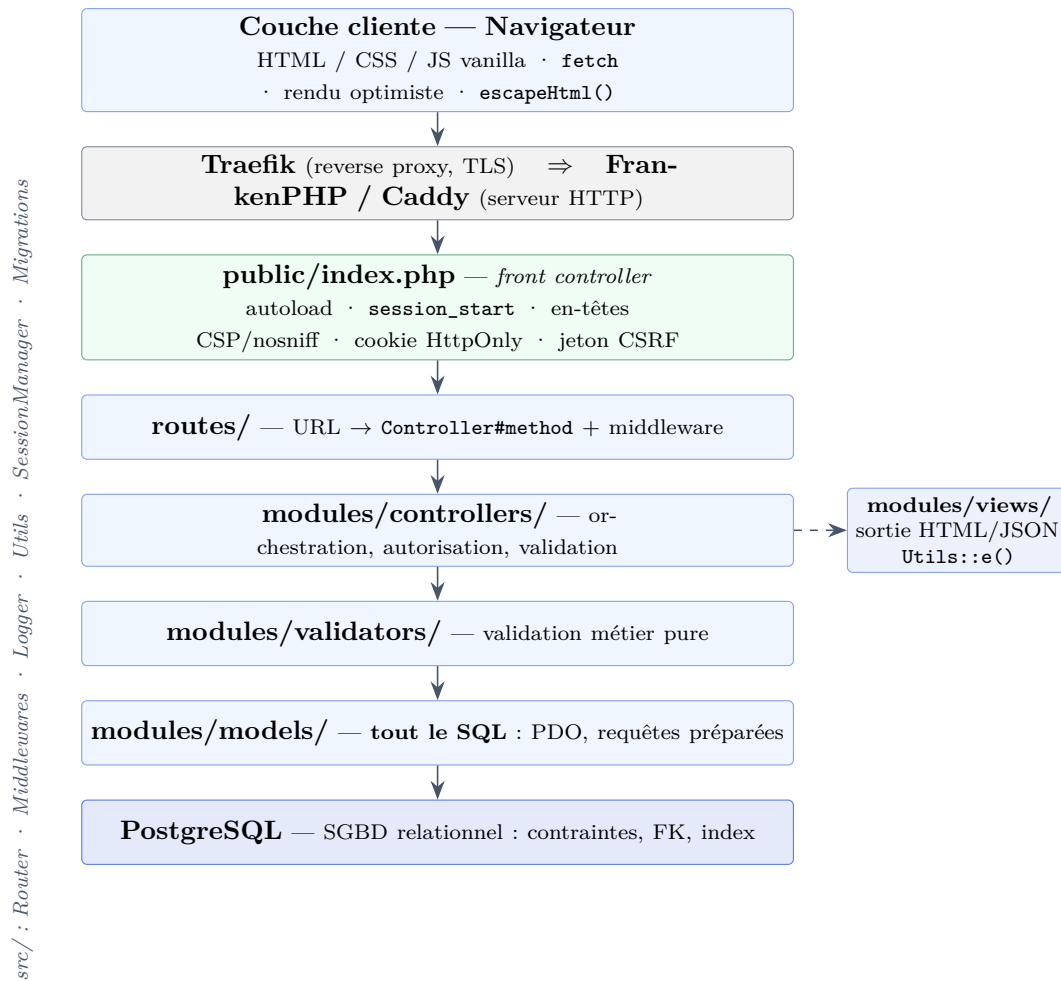
Le découpage est imposé par des règles strictes que je me suis fixées, vérifiables à la lecture du code :

- **aucun SQL hors de `modules/models/`** : un contrôleur appelle une méthode de modèle, il n’écrit jamais de requête ;
- **aucune sortie (`echo`) hors de `modules/views/` et de l’entrée `public/index.php`** ;
- **aucun accès à `$_POST` / `$_GET` / `$_SESSION` hors des contrôleurs et de `SessionManager`** ;
- **aucun appel direct à `Database::getConnection()` hors des modèles**, de Migrations et du *bootstrap*.

Le dossier `src/` regroupe le code « framework » (le routeur, les middlewares, le logger, les utilitaires, le gestionnaire de session, les migrations), distinct du code « métier » (`modules/`). Cette séparation entre infrastructure réutilisable et domaine applicatif est un marqueur d’architecture mûre.

#### 4.1.2 Rôle de sécurité de chaque couche (DICP)

La stratégie de sécurité se lit couche par couche selon les quatre indicateurs de l’ANSSI : **D**isponibilité, **I**ntégrité, **C**onfidentialité, **P**reuve.



**Figure 4.1** — Architecture multicouche de SOLAGE : du navigateur au SGBD.

#### 4.1.3 Patrons mis en œuvre

L'architecture mobilise plusieurs patrons de conception : *Front Controller* (`public/index.php`, point d'entrée unique), *MVC* (séparation modèle / vue / contrôleur), *Middleware* (chaîne de traitement transverse pour l'authentification, l'autorisation et le CSRF) et *Injection de dépendance* (le `SessionManager` reçoit son `UserModel` par constructeur, ce qui le rend testable).

| Couche                                  | Mécanisme de sécurité                                                                                 | Indicateur DICP                    |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------|
| Edge (Traefik)                          | TLS / HTTPS, HSTS, isolation réseau                                                                   | Intégrité, Confidentialité         |
| Bootstrap<br>( <code>index.php</code> ) | En-têtes CSP / nosniff, cookie <code>HttpOnly</code><br>/ <code>Secure</code> / <code>SameSite</code> | Intégrité, Confidentialité         |
| Middleware (src/)                       | Jetons CSRF, authentification, autorisation ; refus journalisé                                        | Intégrité, Confidentialité, Preuve |
| Contrôleurs                             | Validation des entrées, contrôle d'accès par ressource (anti-IDOR)                                    | Intégrité, Confidentialité         |
| Modèles (PDO)                           | Requêtes préparées (anti-injection SQL), transactions                                                 | Intégrité                          |
| Vues                                    | Échappement systématique <code>Utils::e()</code> (anti-XSS)                                           | Intégrité                          |
| SGBD<br>(PostgreSQL)                    | Contraintes, clés étrangères, index, comptes & droits                                                 | Disponibilité, Intégrité           |

Table 4.1 – Rôle de sécurité de chaque couche, selon les indicateurs DICP.

## 4.2 Gestion de projet

J'ai réalisé **seul** l'intégralité de ce projet. Je l'ai mené selon une démarche **itérative légère** (proche de Kanban), en deux temps : une **construction initiale** (2024), puis une **reprise** dédiée à la sécurisation et à l'industrialisation (2026 : conteneurisation, migration PostgreSQL, sécurité, refonte MVC). Sur un projet de cette taille, j'ai assumé l'ensemble des rôles : planification, suivi et qualité.

**Planification.** J'utilise le `README.md` du dépôt comme **tableau Kanban** (colonnes *À faire*, *En cours*, *À finaliser*, *Terminé*) : il me sert de *backlog* de référence et hiérarchise les priorités au regard du risque par bloc de compétences.

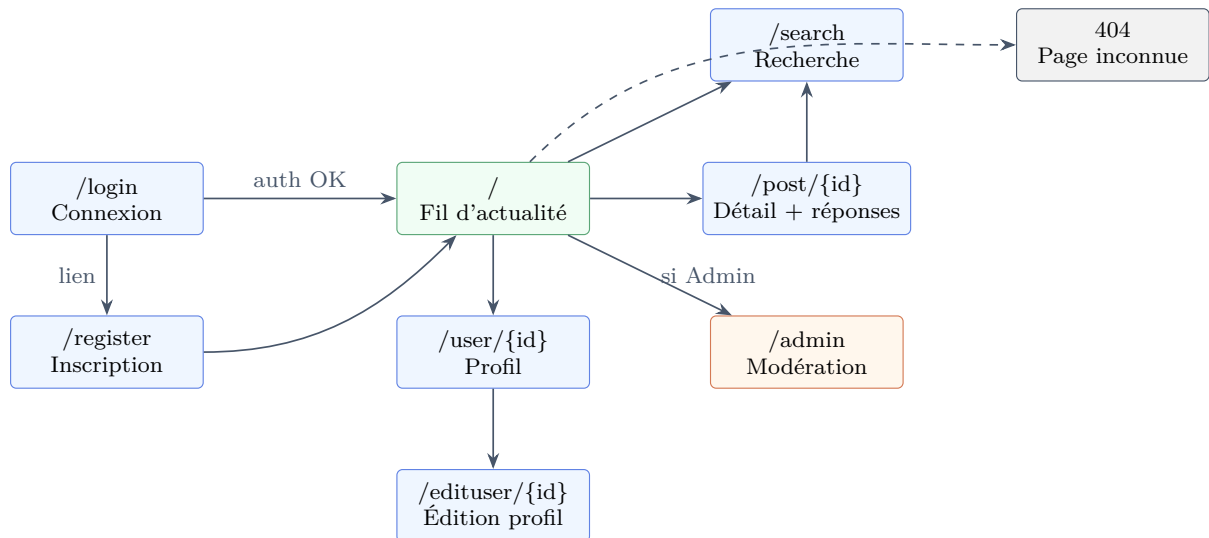
**Suivi des tâches.** J'ai suivi mes tâches au fil des commits **Git** — chaque tâche correspond à un ou plusieurs commits datés — et j'en ai consolidé la trace dans un fichier `SUIVI.md` (tâche, phase, date). Cette trace, vérifiable, me tient lieu de tableau de bord et permet de rapprocher le réalisé du planifié.

**Qualité.** Je me suis fixé des règles de qualité — séparation stricte des couches, requêtes préparées systématiques, échappement systématique des sorties — et je les ai fait respecter en me relisant à chaque fonctionnalité, comme une revue de code à une personne.

**Journal de décisions.** Un journal des décisions techniques (`Probleme-Solution.md`) note, en quelques phrases courtes, chaque problème rencontré et la solution retenue. Par exemple : « *Migrations.php relançait les migrations à chaque requête → cycle de requêtes inutile ; passage à un script `bin/migrate.php` lancé une seule fois au démarrage.* » Le détail du suivi par phases figure en annexe [A.1](#).

### 4.3 Maquettes et enchaînement des écrans

L'application comporte neuf écrans : connexion, inscription, fil d'actualité, détail d'un message, profil utilisateur, édition de profil, recherche, administration et page d'erreur 404. Leur enchaînement est formalisé par la figure 4.2.



**Figure 4.2** – Enchaînement des écrans (storyboard). En vert l'écran protégé par authentification, en orange l'écran réservé à l'administrateur.

J'ai délibérément réutilisé les **conventions d'interface éprouvées** du microblogging — fil chronologique, zone de composition en haut, réponses imbriquées, action « j'aime » sous chaque message — plutôt que d'inventer une interface nouvelle. Ces motifs étant déjà familiers aux utilisateurs, les reprendre maximise l'**apprenabilité** et sert les principes ergonomiques de simplicité et de minimalité des affichages. Ma contribution de conception tient dans l'**adaptation** : un périmètre réduit et cohérent, une charte graphique minimaliste propre à Solage, et l'enchaînement des écrans présenté ci-dessus. Le parcours nominal mène de la connexion au fil d'actualité, depuis lequel l'utilisateur accède au détail d'un message, aux profils, à la recherche et — s'il est administrateur — à la modération.

J'ai formalisé ces écrans sous forme de **maquettes**, réalisées avec **Penpot** (outil de maquettage *open source*). Ce sont mes propres écrans — identité visuelle, charte et libellés propres à SOLAGE, sans aucun élément de marque tiers. Chaque écran est décliné en version **bureau** et **mobile**, actant une conception *responsive*. Les trois écrans les plus structurants sont présentés ici ; les autres figurent en annexe A.5.

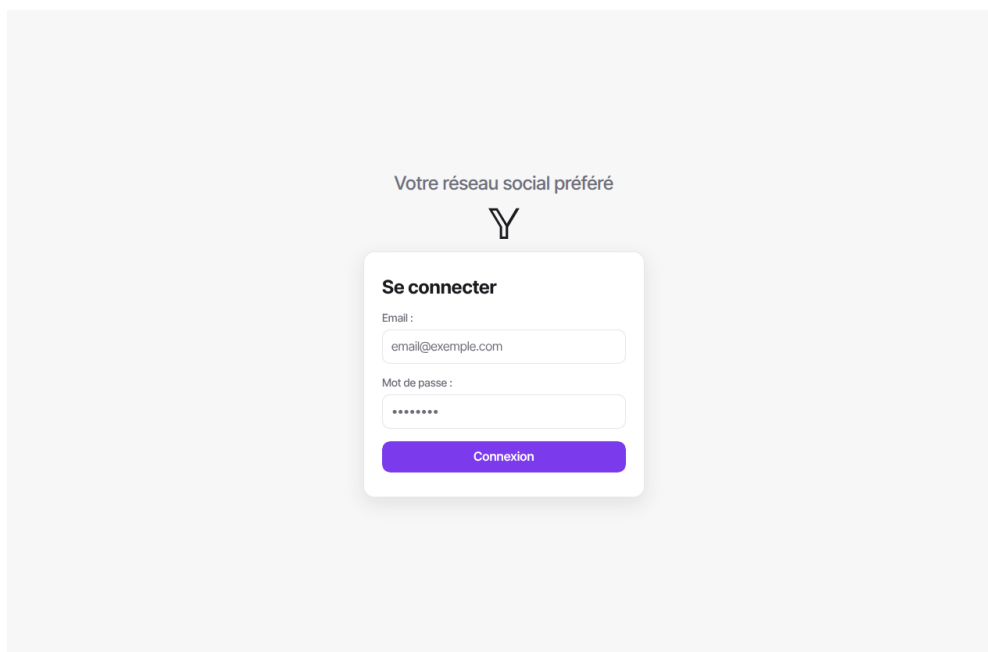


Figure 4.3 — Maquette de l'écran de connexion (bureau).

**Connexion — focaliser sur une tâche unique.** L'écran de connexion isole sa seule tâche dans une **carte centrée** : logo et accroche au-dessus, deux champs étiquetés explicitement (« Email », « Mot de passe »), et un **unique appel à l'action** mis en avant par la couleur d'accent violette. L'absence de tout élément concurrent réduit la charge cognitive et guide l'œil vers l'action attendue.

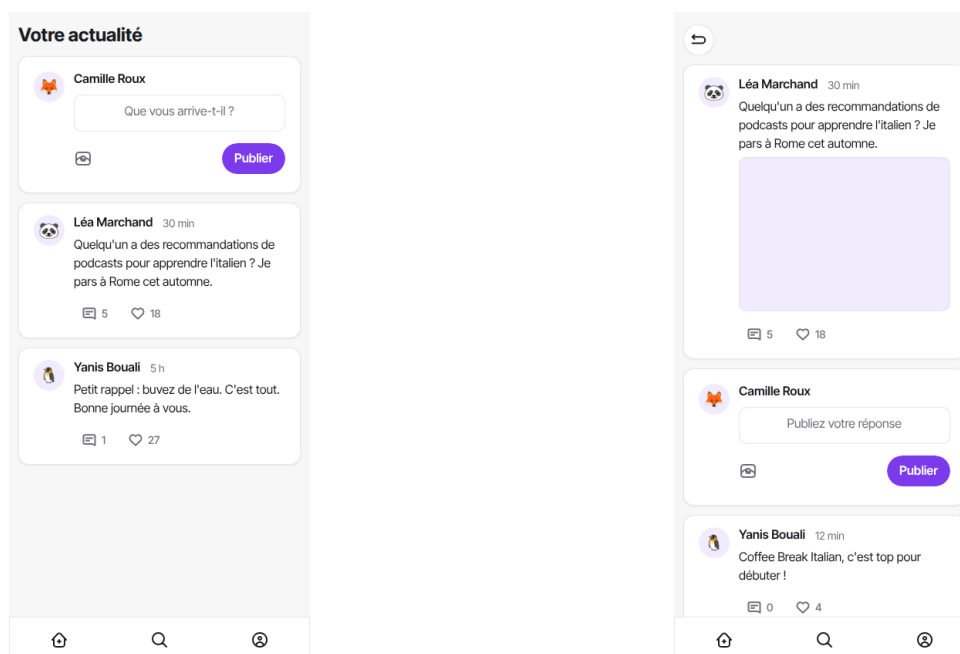


Figure 4.4 — Maquettes mobiles : le fil d'actualité (à gauche) et le détail d'un message avec sa zone de réponse (à droite).

**Fil et détail — hiérarchie visuelle et constance.** Sur le fil comme sur le détail, la **zone de composition** reste en haut, immédiatement accessible. Chaque message suit la même **hiérarchie**



**visuelle** : nom de l’auteur en gras, horodatage atténué en gris, contenu en corps de texte, puis les actions (« répondre », « j’aime ») alignées sous le message. Sur mobile, une **barre de navigation inférieure** (accueil, recherche, profil) reste constante d’un écran à l’autre, et un bouton de retour ramène du détail vers le fil — deux repères de navigation qui ne bougent jamais.

**Choix d’accessibilité (RGAA).** Les maquettes intègrent les repères d’accessibilité visés par le projet : libellés de formulaire explicites associés à chaque champ, contraste suffisant entre le texte et le fond, et zones d’action (boutons, icônes) dimensionnées pour le tactile. Ces choix de conception se traduisent dans le code par des attributs `alt`, `aria-label` et des libellés `<label>` (cf. chapitre 4).

## 4.4 Conception de la base de données

### 4.4.1 Modèle conceptuel des données (MCD)

Le modèle conceptuel (figure 4.5) recense trois entités principales : UTILISATEUR, ROLE et POST. Les réponses sont modélisées par une association *réflexive* « répond à » sur POST : une réponse *est* un message qui en référence un autre. Les « j’aime » et les favoris sont des associations entre UTILISATEUR et POST, l’association « aime » portant l’attribut `created_at`.

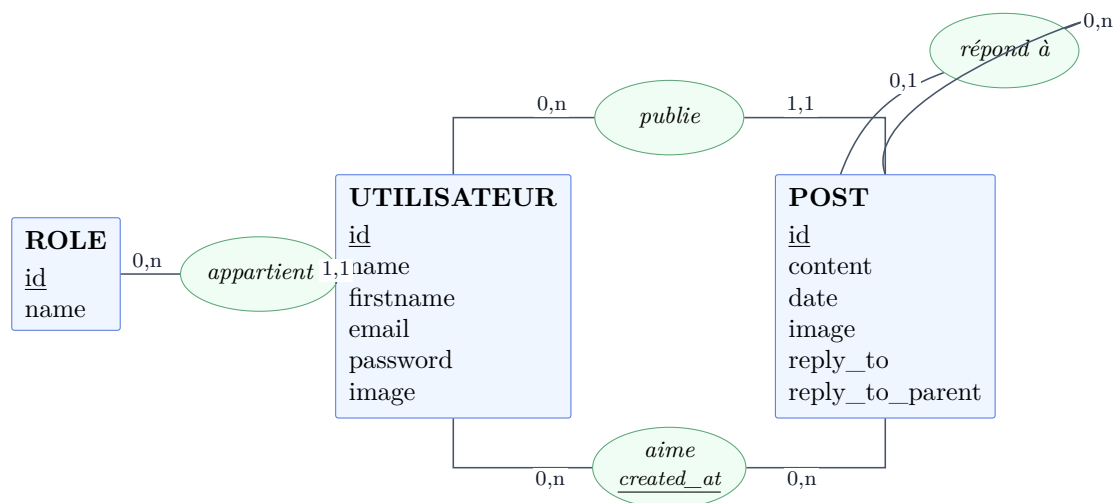


Figure 4.5 – Modèle conceptuel des données (formalisme Merise).

**Les réponses, sans table dédiée.** Une réponse n’est pas un type d’objet distinct : c’est un `post` qui en référence un autre via `reply_to` (le message auquel on répond) et `reply_to_parent` (la racine du fil). Ce choix évite une table `responses` séparée qui dupliquerait la structure d’un message, et permet de traiter messages et réponses avec le même modèle et les mêmes requêtes.

### 4.4.2 Modèle logique / physique (MLD, MPD)

Le passage au modèle logique transforme les associations en clés étrangères. La règle de nommage notable : le mot `user` étant *réservé* en PostgreSQL, les colonnes de clé étrangère se nomment `user_id`. La figure 4.6 présente le schéma relationnel et ses clés étrangères.

Le modèle physique est concrétisé par le script de création `solage.pg.sql`, chargé une seule fois à l'initialisation du conteneur PostgreSQL. L'extrait suivant en montre les tables principales ; le script complet figure en annexe [A.2](#).

```

1 CREATE TABLE roles (
2     id SERIAL PRIMARY KEY,
3     name VARCHAR(255) NOT NULL
4 );
5
6 CREATE TABLE users (
7     id SERIAL PRIMARY KEY,
8     name VARCHAR(255) NOT NULL,
9     email VARCHAR(255) NOT NULL UNIQUE,
10    password VARCHAR(255) NOT NULL,
11    role INT NULL REFERENCES roles(id) ON DELETE SET NULL,
12    image VARCHAR(255) NULL
13 );
14 CREATE INDEX users_role_idx ON users(role);
15
16 CREATE TABLE posts (
17     id SERIAL PRIMARY KEY,
18     user_id INT NULL REFERENCES users(id) ON DELETE CASCADE,
19     date TIMESTAMP NOT NULL,
20     content TEXT NULL,
21     reply_to INT NULL,
22     reply_to_parent INT NULL,
23     image TEXT NULL
24 );
25 CREATE INDEX posts_user_idx ON posts(user_id);
26
27 CREATE TABLE likes (
28     id SERIAL PRIMARY KEY,
29     user_id INT NULL REFERENCES users(id) ON DELETE CASCADE,
30     post INT NULL REFERENCES posts(id) ON DELETE CASCADE,
31     created_at TIMESTAMP NOT NULL
32 );

```

**Listing 4.1** – Extrait du script de création — `solage.pg.sql`

L'intégrité est garantie par les contraintes : NOT NULL, UNIQUE(email), clés étrangères avec ON DELETE CASCADE (suppression en cascade des messages d'un utilisateur supprimé) ou SET NULL. Des **index** accélèrent les accès fréquents (clés étrangères, fil d'actualité).

#### 4.4.3 Évolution du schéma : migrations idempotentes

Les évolutions additives ne sont pas appliquées à la main : elles passent par un système de **migrations idempotentes** (`src/Migrations.php`), qui interroge `information_schema` avant d'agir et peut donc être rejoué sans risque.

```

1 protected function columnExists(string $table, string $column): bool {
2     $sql = "SELECT 1 FROM information_schema.columns

```

```

3         WHERE table_schema = current_schema()
4         AND table_name = :table AND column_name = :column LIMIT 1";
5     $stmt = $this->db->prepare($sql);
6     $stmt->execute([':table' => $table, ':column' => $column]);
7     return $stmt->fetchColumn() !== false;
8 }
9
10 public function migrate(): void {
11     $this->addColumnIfNotExists('posts', 'reply_to', 'INT', 'NULL');
12     $this->addColumnIfNotExists('posts', 'reply_to_parent', 'INT', 'NULL');
13     $this->addColumnIfNotExists('posts', 'image', 'TEXT', 'NULL');
14 }

```

Listing 4.2 – Migration idempotente — src/Migrations.php

**Jeu d'essai et restauration.** Un jeu d'essai reproductible (`seed.sql`) peuple une base de test (utilisateurs, messages, « j'aime », réponses).

**Sauvegarde et restauration de la base.** La sauvegarde de la base de production est automatisée par le script `scripts/backup.sh`, planifié en *cron* (une fois par jour). Il exécute `pg_dump` à l'intérieur du conteneur PostgreSQL — les identifiants ne transitent jamais par la ligne de commande, ils sont lus dans l'environnement du conteneur — et écrit un *dump* compressé et horodaté dans `backups/`. L'option `--clean --if-exists` rend la restauration rejouable (suppression puis recréation des objets), un renommage atomique garantit qu'un *dump* partiel n'est jamais conservé sous le nom final, et les sauvegardes de plus de 14 jours sont purgées (rotation). Les *dumps* restent hors du dépôt Git (`backups/` est dans `.gitignore`) : ils contiennent des données.

La restauration sur une base vierge — exigée par le critère C7 (« la base de test peut être restaurée en cas d'incident ») — réinjecte le *dump* décompressé dans `psql` :

```

1 gunzip -c backups/solage-AAAA-MM-JJ-HHMMSS.sql.gz | \
2 docker compose -f docker-compose.prod.yml exec -T postgres \
3 sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"'

```

Listing 4.3 – Restauration d'un *dump* compressé dans le conteneur

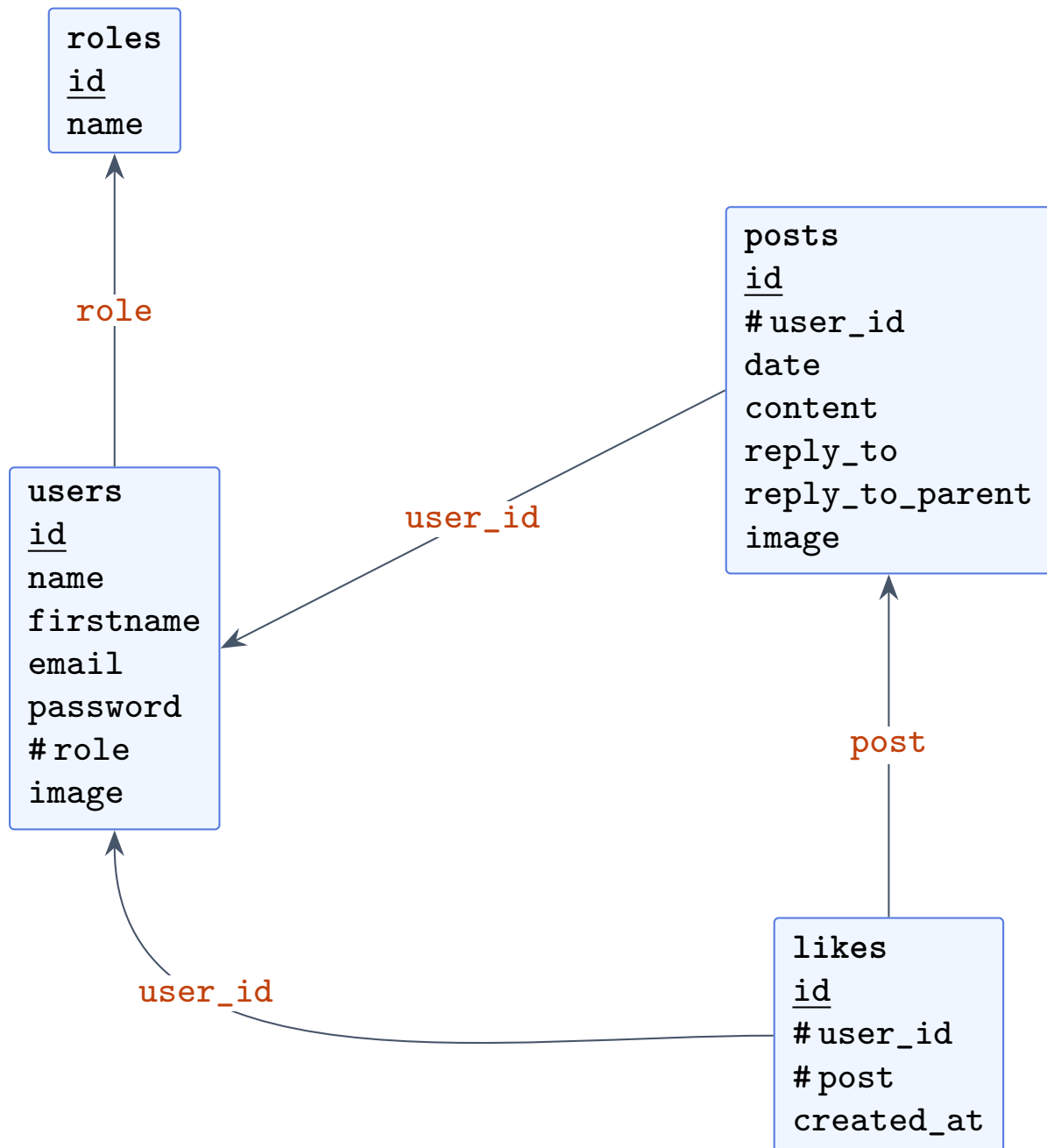


Figure 4.6 — Modèle logique des données : tables physiques et clés étrangères (#).

## 4.5 Diagrammes UML

### 4.5.1 Diagramme de cas d'utilisation

La figure 4.7 formalise le comportement attendu : les trois acteurs et leurs cas d'usage, avec la généralisation  $\text{Administrateur} \rightarrow \text{Utilisateur} \rightarrow \text{Visiteur}$ .

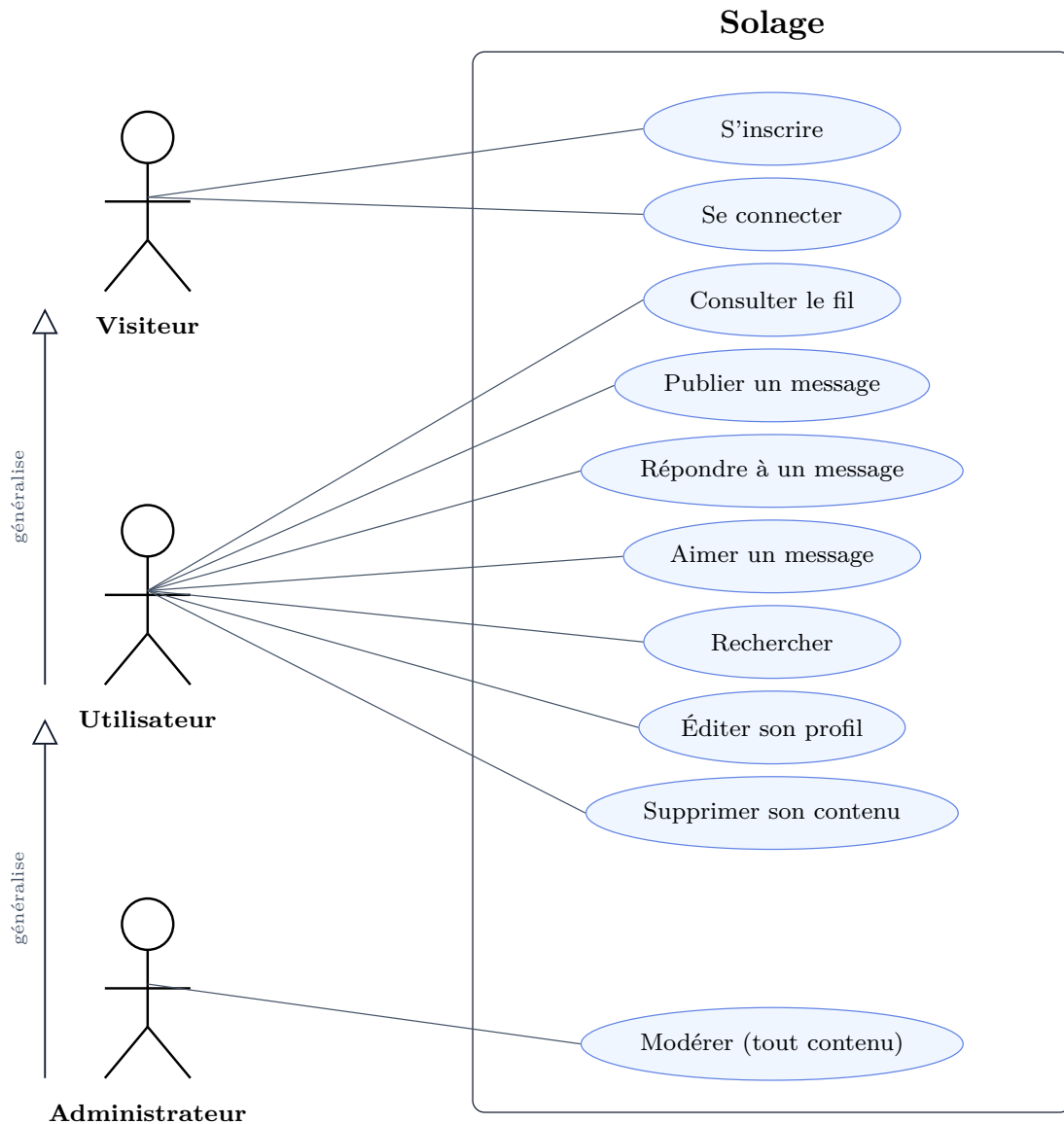
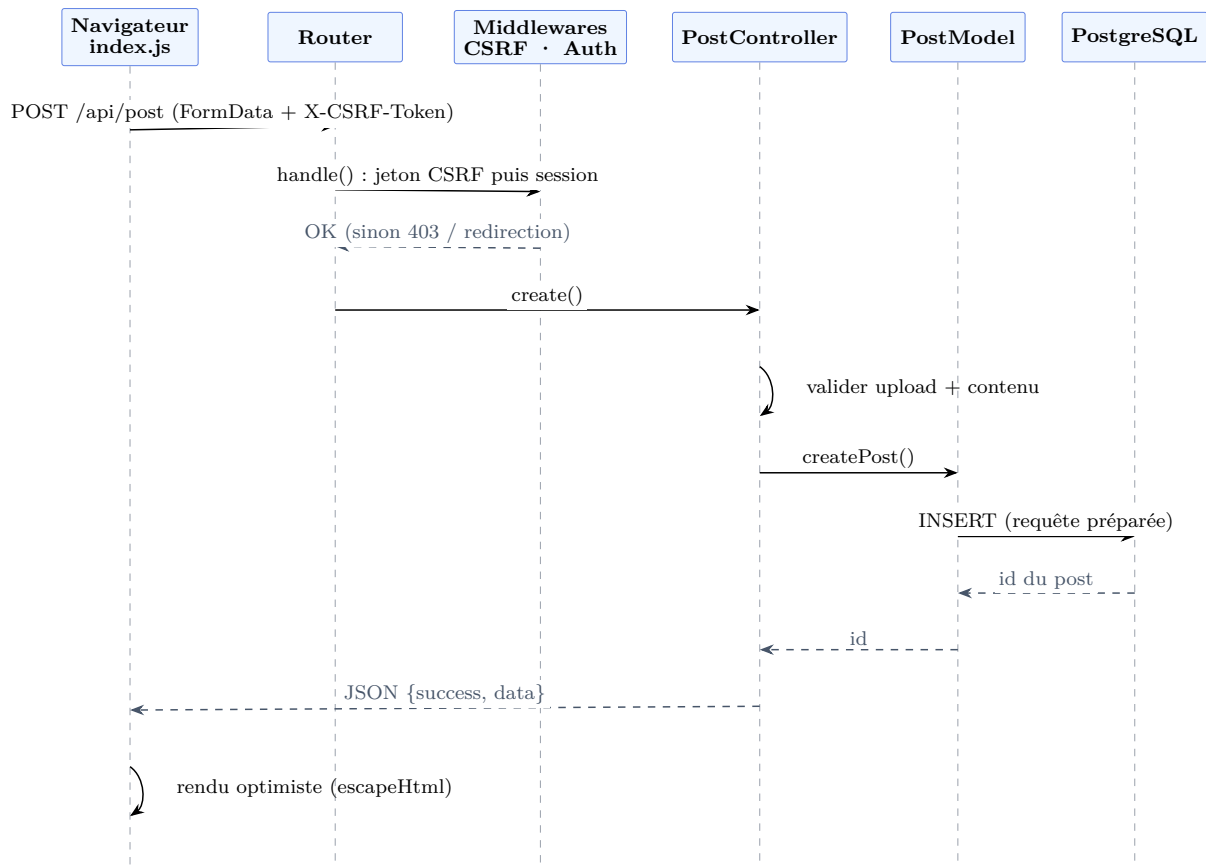


Figure 4.7 – Diagramme de cas d'utilisation de SOLAGE.

### 4.5.2 Diagramme de séquence : « Publier un message »

Le cas le plus significatif est la publication d'un message : il traverse toutes les couches et mobilise les protections de sécurité. La figure 4.8 en détaille le déroulé, de l'appel `fetch` du navigateur jusqu'à l'insertion en base et au rendu optimiste.



**Figure 4.8** – Diagramme de séquence : publication d’un message (avec passage par les middlewares CSRF et authentification).

### 4.5.3 Diagramme de classes (couche modèle)

La figure 4.9 présente un extrait de la couche modèle et le gestionnaire de session, avec leurs principales méthodes et dépendances.

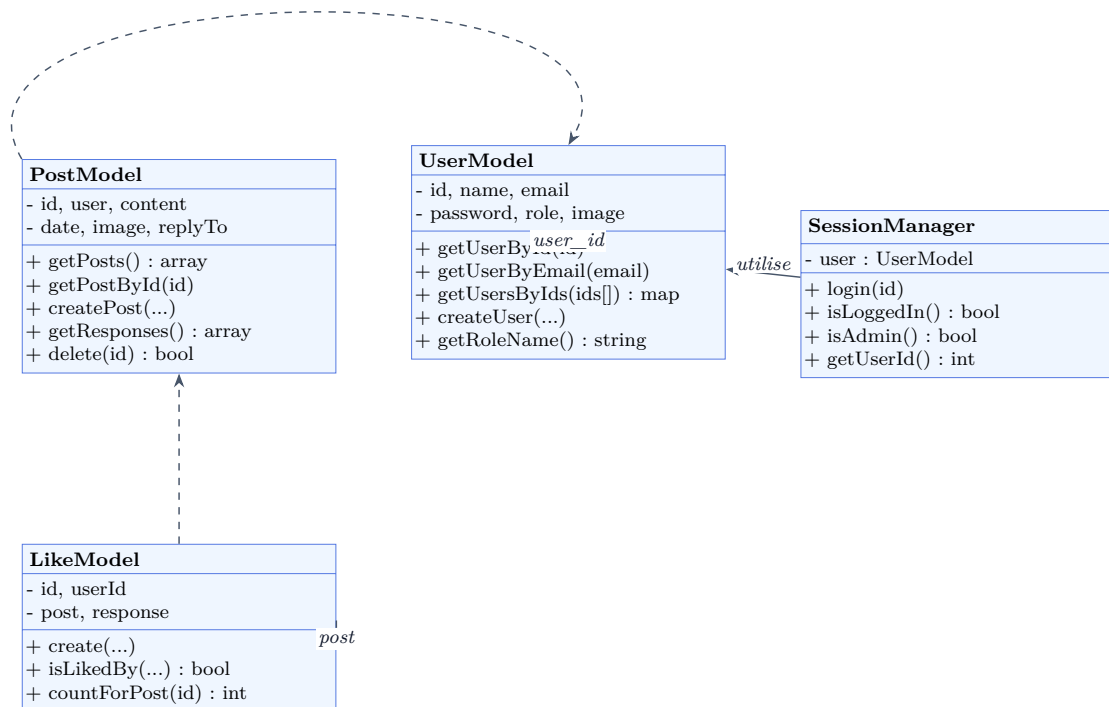


Figure 4.9 – Diagramme de classes (extrait) : modèles et gestion de session.

## 4.6 Composants d'interface utilisateur

Les interfaces sont rendues par la couche modules/views/. Une vue ne contient aucune logique d'accès aux données : elle reçoit des données déjà préparées par le contrôleur et se contente de produire du HTML, en **échappant systématiquement** tout contenu issu de l'utilisateur via `Utils::e()`.

```

1 <div class="createPost">
2   <div class="postAvatar"><?= Utils::e($userImage) ?></div>
3   <div class="postInsideContainer">
4     <div class="postNameDate"><?= Utils::e($username) ?></div>
5     <span id="postContent" aria-label="Contenu du post"
6       role="textbox" contenteditable="true"></span>
7     <input id="file-input" accept="image/*" type="file" class="hidden"
8       />
9     <button id="postCreateButton" class="postCreateButton">Publier </
10    button>
11  </div>
12 </div>

```

Listing 4.4 – Vue du formulaire de publication (extrait) — modules/views/CreatePostView.php

**Asynchronisme (AJAX).** Les actions (publier, aimer, supprimer, se connecter) sont réalisées sans rechargement de page, par `fetch`. Après création d'un message, le DOM est mis à jour de façon *optimiste* : le message est injecté immédiatement, sans attendre un rechargement. Comme cette injection passe par `innerHTML`, un échappement *côté client* (`escapeHtml`), miroir

de `Utils::e()`, est indispensable (voir la section 4.10).

```
1  const content = document.getElementById("postContent").innerText.trim();
2  if (!content) { alert("Le contenu du post ne peut pas être vide !"); return;
   }
3
4  const formData = new FormData();
5  formData.append('data', JSON.stringify({ content, replyTo, replyToParent }))
   ;
6  if (selectedImage) formData.append('image', selectedImage);
7
8  fetch("/api/post", { method: "POST", body: formData }) // le jeton CSRF est
   injecté
9  .then(response => response.json()) // automatiquement (
   cf. ch. securite)
10 .then(data => { if (data.success) { /* rendu optimiste via escapeHtml */ }
   });
```

**Listing 4.5** – Création d'un message par fetch — `public/scripts/index.js`

La figure 4.10 montre l'interface correspondante : le fil d'actualité et, en tête, le formulaire de publication d'un message (« Que vous arrive-t-il ? »), dont le code JavaScript ci-dessus assure l'envoi asynchrone.





**Figure 4.10** – Fil d’actualité et formulaire de publication (vue mobile). Le bouton « Publier » déclenche le `fetch POST /api/post` détaillé ci-dessus.

## 4.7 Composants métier

Les composants métier vivent dans `modules/controllers/`. Ils orchestrent la requête, valident les entrées et — point essentiel — vérifient l’**autorisation sur la ressource**, et non la seule authentification.

### 4.7.1 Contrôle d’accès : la parade à l’IDOR

La suppression d’un message illustre le contrôle d’accès. Avant toute suppression, le contrôleur vérifie que l’utilisateur courant est *propriétaire* de la ressource ou administrateur ; sinon, il répond 403 et journalise la tentative.

```

1 $session = new SessionManager(new UserModel());
2 $userId  = $session->getUserId();
3 $post    = PostModel::getPostById($postId);
4
```

```

5  if ($post->getUserId() !== $userId && !$session->isAdmin()) {
6      Logger::get()->warning('post.delete.forbidden', [
7          'current_user_id' => $userId,
8          'post_owner_id'   => $post->getUserId(),
9          'post_id'         => $postId,
10     ]);
11     http_response_code(403);
12     Utils::sendResponse(false, "Vous n'avez pas la permission de supprimer
    ce post.");
13     return;
14 }
15 $result = PostModel::delete($postId);

```

Listing 4.6 – Contrôle d'autorisation anti-IDOR — modules/controllers/PostController.php

#### 4.7.2 Authentification vs autorisation

La distinction est portée par le `SessionManager` : `isLoggedIn()` répond à « qui es-tu ? » (authentification), `isAdmin()` répond à « qu'as-tu le droit de faire ? » (autorisation, résolue par le *libellé* du rôle et non par un identifiant numérique magique).

```

1  public function isLoggedIn(): bool {
2      return isset($_SESSION['user_id']);
3  }
4  public function isAdmin(): bool {
5      return $this->user !== null && $this->user->getRoleName() === 'Admin';
6  }

```

Listing 4.7 – Authentification et autorisation — src/SessionManager.php

#### 4.7.3 Validation pure, découplée de la sortie HTTP

Le validateur (modules/validators/UserValidator.php) *décide* sans produire de sortie : il retourne un tableau structuré. C'est le contrôleur qui décide ensuite quoi en faire. Ce découplage rend le validateur **testable** (il n'émet ni `echo` ni en-tête).

```

1  public static function login($email, $password): array {
2      if (empty($email) || empty($password)) {
3          return ['ok' => false, 'type' => 'error', 'message' => "Merci de
    renseigner vos informations"];
4      }
5      $user = (new UserModel())->getUserByEmail($email);
6      if (!$user) {
7          return ['ok' => false, 'type' => 'error', 'message' => "L'email n'
    existe pas"];
8      }
9      if (!password_verify($password, $user->getPassword())) {
10         return ['ok' => false, 'type' => 'error', 'message' => "Le mot de
    passe ne correspond pas"];
11     }

```

```

12     return ['ok' => true, 'type' => 'success', 'message' => "Vous vous êtes
        bien connecté !"];
13 }

```

Listing 4.8 — Validation pure (retourne un tableau) — UserValidator.php

## 4.8 Composants d'accès aux données

Toute la persistance vit dans `modules/models/`. Les requêtes sont **préparées** : les valeurs utilisateur partent en paramètres liés, jamais concaténées dans la requête — l'injection SQL est structurellement impossible.

```

1  public function createPost(?int $replyTo = null, ?int $replyToParent = null)
    {
2      try {
3          $statement = $this->db->prepare(
4              'INSERT INTO posts (user_id, content, date, reply_to, image,
                reply_to_parent)
5              VALUES (:user_id, :content, :date, :reply_to, :image, :
                reply_to_parent)');
6          $statement->bindValue(':user_id', $this->user);
7          $statement->bindValue(':content', $this->content);
8          $statement->bindValue(':date', $this->date);
9          $statement->bindValue(':reply_to', $replyTo, PDO::PARAM_INT);
10         $statement->bindValue(':image', $this->image);
11         $statement->bindValue(':reply_to_parent', $replyToParent, PDO::
            PARAM_INT);
12         $statement->execute();
13         return $this->db->lastInsertId('posts_id_seq');
14     } catch (PDOException $e) {
15         Logger::get()->error('post.create.failed', ['user_id' => $this->user
            , 'exception' => $e]);
16         return false;
17     }
18 }

```

Listing 4.9 — Insertion par requête préparée — modules/models/PostModel.php

**Optimisation : suppression d'une requête N+1.** Afficher l'auteur de chaque message provoquait, à l'origine, une requête par message (21 requêtes pour 20 messages). La méthode `getUsersByIds` précharge tous les auteurs en *une* requête `IN`, ramenant le coût à deux requêtes. Le cas `IN ()` vide — une erreur SQL en PostgreSQL — est court-circuité.

```

1  public static function getUsersByIds(array $ids): array {
2      if (empty($ids)) {                                     // IN () est une erreur SQL en
        PostgreSQL
3          return [];
4      }
5      $placeholders = implode(',', array_fill(0, count($ids), '?'));

```

```

6      $stmt = Database::getConnection()->prepare(
7          "SELECT id, name, email, password, role, image FROM users WHERE id
            IN ($placeholders)");
8      $stmt->execute(array_values($ids));    // valeurs liées : pas d'
            injection possible
9      $users = [];
10     while ($row = $stmt->fetch(PDO::FETCH_OBJ)) {
11         $users[(int)$row->id] = new UserModel($row);
12     }
13     return $users;
14 }

```

Listing 4.10 — Préchargement des auteurs en une requête — modules/models/UserModel.php

## 4.9 Autres composants : le socle « framework »

Le dossier `src/` contient le socle que j'ai écrit : routeur, middlewares, logger, utilitaires. C'est la preuve de maîtrise architecturale — chaque ligne est défendable.

**Routeur.** Le routeur associe une URL (avec paramètres en expression régulière) à un `Controller#method`, exécute le middleware de route, puis **arme le contrôle CSRF sur toute requête POST** — sécurité par défaut plutôt que sur option.

```

1  if ($route['method'] === $requestMethod && preg_match($pathRegex,
    $requestUri, $matches)) {
2      if ($route['middleware']) {
3          (new $route['middleware']())->handle();    // Auth ou Admin, selon
            la route
4      }
5      if ($route['method'] === 'POST') {
6          (new CsrMiddleware())->handle();    // CSRF armé sur
            TOUTES les mutations
7      }
8      // ... instantiation du contrôleur et appel de la méthode ...
9  }

```

Listing 4.11 — Dispatch et armement du CSRF sur les POST — src/Router.php

**Middlewares : authentification vs autorisation.** Deux classes distinctes séparent explicitement « être connecté » et « être administrateur ». Le refus d'accès administrateur est journalisé (indicateur de *preuve* du DICP).

```

1  public function handle() {
2      $session = new SessionManager(new UserModel());
3      if (!$session->isLoggedIn()) { header('Location: /login'); exit; }
4      if (!$session->isAdmin()) {
5          Logger::get()->warning('admin.access.denied', [
6              'user_id' => $session->getUserId(), 'uri' => $_SERVER['
                REQUEST_URI'] ?? '');

```

```

7         http_response_code(403);
8         echo "403 -- accès réservé aux administrateurs."; exit;
9     }
10 }
```

Listing 4.12 – Middleware d'autorisation administrateur — src/AdminMiddleware.php

**Journalisation PSR-3.** Le logger (src/Logger.php) implémente le standard PSR-3 et écrit des enregistrements *JSON-line* sur la sortie standard (et la sortie d'erreur pour les niveaux **warning** et au-delà), conformément aux conventions Docker : c'est l'orchestrateur qui collecte les journaux, l'application ne gère pas de fichiers de log.

```

1 public function log($level, string|Stringable $message, array $context =
    []): void {
2     $record = ['ts' => date('c'), 'level' => (string) $level,
3               'msg' => $this->interpolate((string) $message, $context)];
4     if ($context !== []) { $record['context'] = $this->serializeContext(
5         $context); }
6     // STDOUT/STDERR n'existent qu'en CLI ; les flux php:/* marchent
7     // partout (CLI, HTTP, FrankenPHP)
8     $target = in_array($level, self::STDERR_LEVELS, true) ? 'php://stderr' :
9               'php://stdout';
10    file_put_contents($target, json_encode($record, JSON_UNESCAPED_UNICODE)
11        . "\n", FILE_APPEND);
12 }
```

Listing 4.13 – Logger PSR-3, sortie JSON sur php://stdout — src/Logger.php

## 4.10 Sécurité de l'application

La sécurité est traitée **faillie par faille**, en référence à l'OWASP Top 10 et aux recommandations de l'ANSSI. Le tableau 4.2 résume les menaces couvertes et leur parade dans SOLAGE.

| Menace (OWASP)                | Parade dans Solage                                                                               | Fichier(s)                     |
|-------------------------------|--------------------------------------------------------------------------------------------------|--------------------------------|
| XSS stocké (A03)              | Échappement à la sortie : <code>Utils::e()</code> (serveur) + <code>escapeHtml()</code> (client) | vues, index.js                 |
| CSRF                          | Jeton de synchronisation armé sur tout POST, comparaison à temps constant                        | CsrfMiddleware, CsrfHelper     |
| Injection SQL (A03)           | Requêtes préparées PDO partout                                                                   | modules/models/                |
| Contrôle d'accès / IDOR (A01) | Vérification « propriétaire ou administrateur » + 403 + journal                                  | PostController, UserController |
| Mots de passe                 | <code>password_hash</code> / <code>password_verify</code> (bcrypt)                               | UserModel                      |
| En-têtes                      | CSP, nosniff, Referrer-Policy, HSTS (prod)                                                       | index.php, prod                |
| Cookie de session             | <code>HttpOnly</code> + <code>SameSite=Lax</code> + <code>Secure</code> (prod)                   | index.php                      |

Table 4.2 – Couverture des principales failles web.

### 4.10.1 XSS : échappement à la sortie

Le choix d'architecture est d'échapper **à la sortie**, pas à l'entrée : on conserve la donnée originale (un utilisateur peut légitimement écrire « < ») et on neutralise au moment de l'affichage. Côté serveur, `Utils::e()` encapsule `htmlspecialchars`; côté client, `escapeHtml()` en est le miroir, requis car le JavaScript reconstruit du DOM.

```
1 public static function e(?string $value): string {
2     return htmlspecialchars($value ?? '', ENT_QUOTES | ENT_HTML5, 'UTF-8');
3 }
```

Listing 4.14 — Échappement côté serveur — `src/Utils.php`

### 4.10.2 CSRF : jeton de synchronisation

La protection CSRF suit le patron *Synchronizer Token* : un secret par session, généré par un générateur cryptographique (`random_bytes`), exposé dans une balise `<meta>` et renvoyé par le client dans l'en-tête `X-CSRF-Token`. La vérification compare en **temps constant** (`hash_equals`), ce qui protège des attaques temporelles.

```
1 public static function getToken(): string {
2     if (empty($_SESSION['csrf_token'])) {
3         $_SESSION['csrf_token'] = bin2hex(random_bytes(32)); // CSPRNG,
4         // pas uniqid()/rand()
5     }
6     return $_SESSION['csrf_token'];
7 }
8
9 public static function verifyToken(?string $token): bool {
10    return !empty($_SESSION['csrf_token'])
11        && hash_equals($_SESSION['csrf_token'], (string) $token); // temps
12    // constant
13 }
```

Listing 4.15 — Génération et vérification du jeton CSRF — `src/CsrfHelper.php`

Le jeton est armé par le routeur sur *toutes* les requêtes POST (section 4.9), et injecté automatiquement dans chaque `fetch` par une enveloppe unique côté client :

```
1 const token = document.querySelector('meta[name="csrf-token"]')?.content;
2 const nativeFetch = window.fetch.bind(window);
3 window.fetch = function (resource, options = {}) {
4     const method = (options.method || 'GET').toUpperCase();
5     const safe = method === 'GET' || method === 'HEAD' || method === '
6     OPTIONS';
7     if (token && !safe) {
8         const headers = new Headers(options.headers || {});
9         headers.set('X-CSRF-Token', token);
10        options = { ...options, headers };
11    }
12    return nativeFetch(resource, options);
13 };
```

---

**Listing 4.16** — Injection automatique du jeton dans tout fetch — public/scripts/index.js

**Une défense en profondeur.** Le cookie de session est déjà en `SameSite=Lax` (protection *navigateur*), et le jeton de synchronisation ajoute une protection *applicative*, indépendante du navigateur. On garde les deux.

### 4.10.3 En-têtes de sécurité et cookie de session

Le point d'entrée `public/index.php` pose les en-têtes applicatifs et durcit le cookie de session *avant* tout démarrage de session (les paramètres du cookie se figent au démarrage).

```

1 session_set_cookie_params([
2     'httponly' => true,                // inaccessible au JS (anti-vol
        de session par XSS)
3     'samesite' => 'Lax',                // anti-CSRF de base
4     'secure'   => APP_ENV === 'production', // HTTPS uniquement en
        production
5 ]);
6 header("X-Content-Type-Options: nosniff"); // anti
    MIME-sniffing
7 header("Referrer-Policy: strict-origin-when-cross-origin"); // pas de
    fuite d'URL
8 header("Content-Security-Policy: default-src 'self'; img-src 'self' data:;")
    ;
9 session_start();

```

**Listing 4.17** — En-têtes et cookie de session — public/index.php

L'en-tête **HSTS** (**Strict-Transport-Security**) est posé à la couche TLS, par Traefik en production, car c'est lui qui termine le HTTPS — et non PHP, qui ne voit que du HTTP interne.

### 4.10.4 IDOR et contrôle d'accès

La parade à l'IDOR (*Insecure Direct Object Reference*, OWASP A01) est détaillée en section 4.7 : un utilisateur ne peut modifier ou supprimer que ses propres ressources. Cette faille — et le cheminement qui a conduit à la corriger — est documentée dans la veille de sécurité (section 4.15).

### 4.10.5 Axes d'amélioration identifiés

L'analyse de sécurité serait incomplète sans nommer ses propres limites. Trois renforcements ont été identifiés et sont assumés comme axes d'amélioration : aucun n'est bloquant pour l'usage visé, mais chacun élèverait le niveau de défense.

— **Validation serveur systématique** : durcir `UserValidator` (format de l'email, robustesse du mot de passe : longueur, majuscule, chiffre). La validation porte aujourd'hui sur la présence des champs et l'unicité de l'email, pas sur leur forme.

- **Anti-fixation de session** : appeler `session_regenerate_id(true)` juste après une connexion réussie, pour qu'un identifiant de session capté avant l'authentification cesse d'être valable.
- **Validation d'upload par type MIME réel** (`finfo` + `getimagesize`) en complément de l'extension, et limitation de la taille des fichiers.

## 4.11 Plan de tests

La stratégie de tests couvre quatre niveaux : tests **unitaires** (composants isolés), tests d'**intégration** (parcours traversant plusieurs couches), tests de **sécurité** (rejouant les failles corrigées) et tests de **non-régression**. Le tableau 4.3 présente le plan, dérivé des fonctionnalités et des *user stories* de la section 2.4.

| Fonctionnalité         | Type de test      | Objet du test                                                                                                         |
|------------------------|-------------------|-----------------------------------------------------------------------------------------------------------------------|
| Échappement (XSS)      | Unitaire          | <code>Utils::e()</code> : balise, guillemets, apostrophe, esperluette, <code>null</code>                              |
| Transport JSON         | Unitaire          | <code>Utils::sendResponse</code> (forme <code>success/message</code> ), <code>Utils::isAjax</code>                    |
| Jeton CSRF             | Unitaire          | <code>CsrfHelper</code> : génération (64 hex, stable), vérification (bon / mauvais / vide / nul)                      |
| Session, autorisation  | Unitaire (mock)   | <code>SessionManager::login</code> / <code>isAdmin</code> — <code>UserModel</code> mocké, sans BDD                    |
| Connexion, inscription | Intégration       | <code>UserValidator</code> + <code>UserModel</code> : login, register, mot de passe haché                             |
| Accès données          | Intégration       | <code>PostModel</code> : création, lecture, suppression                                                               |
| Injection SQL          | Sécurité (intég.) | <code>SearchModel</code> : charge ' OR '1'='1 rendue inerte                                                           |
| CSRF (bout-en-bout)    | Fonctionnel       | POST sans jeton → 403 + journal                                                                                       |
| IDOR (bout-en-bout)    | Fonctionnel       | Suppression du message d'autrui → 403 + journal                                                                       |
| XSS (bout-en-bout)     | Fonctionnel       | Message <code>&lt;script&gt;</code> stocké brut, rendu échappé                                                        |
| Routage                | Non automatisé    | <code>Router::match</code> lit <code>\$_SERVER</code> / <code>exit</code> ; refactor <code>resolve()</code> identifié |

Table 4.3 – Plan de tests de SOLAGE.

**Environnement de test.** La compétence C9 est implémentée avec **PHPUnit 11** (dépendance de développement Composer). Le *bootstrap tests/bootstrap.php* réutilise l'**autoloader maison** de production (*includes/autoload.php*) : les classes de l'application sont chargées exactement comme par *public/index.php*, sans autoloader de test parallèle. Les tests purs et au *mock* s'exécutent sans base ; les tests d'intégration visent la **vraie base PostgreSQL**, chaque test tournant dans une **transaction annulée** au *teardown* : la base est jetable, son état identique avant et après la suite.

**La testabilité découle du design.** Un composant n'est testable unitairement que si ses dépendances sont substituables. `SessionManager` reçoit son `UserModel` **par injection** : en test on lui passe un *mock*, donc `login()` et `isAdmin()` se vérifient **sans base**. À l'inverse, `UserValidator`



instancie son modèle en dur (`new UserModel()`) : non *mockable*, il est testé **en intégration** contre la base réelle. Ce contraste est un choix assumé.

| Niveau            | Composants couverts                                                                                                                                     | Cas |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Unitaire pur      | Utils (échappement, AJAX, réponse JSON), <code>CsrfHelper</code> (jeton, vérification)                                                                  | 19  |
| Unitaire au mock  | <code>SessionManager</code> ( <code>login</code> , <code>isAdmin</code> ) — injection de dépendance                                                     | 3   |
| Intégration       | <code>UserModel</code> + <code>UserValidator</code> (lecture, hachage, connexion, inscription); <code>PostModel</code> (création, lecture, suppression) | 16  |
| Sécurité (intég.) | Injection SQL sur <code>SearchModel</code> : charge inerte / terme normal                                                                               | 2   |

**Table 4.4** – Familles de tests automatisés (40 cas, 62 assertions).

**Résultats d'exécution.** La suite complète passe au vert en une commande :

```
PHPUnit 11.0.0 by Sebastian Bergmann and contributors.
Runtime: PHP 8.2.0

..... 40 / 40 (100%)

Csrf Helper      8      Post Model      6      Session Manager  3
Sql Injection    2      User Model    10      Utils           11

OK (40 tests, 62 assertions)
```

**Listing 4.18** – Résumé d'exécution de la suite PHPUnit (`-testdox`)

**Tests de sécurité fonctionnels (CSRF, IDOR).** Certaines protections vivent au niveau HTTP (lecture de `$_SERVER`, `php://input`, `exit`) et ne s'automatisent pas proprement en PHPUnit ; elles sont validées **fonctionnellement**, application lancée, la preuve étant le **statut HTTP** et la **ligne de journal** :

- **CSRF** — un POST `/login` sans jeton renvoie 403 (« Token CSRF refusé. ») et journalise `csrf.token.rejected` ;
- **IDOR** — connecté en *Bob* (modérateur), une tentative de suppression d'un message d'*Alice* renvoie 403 et journalise `post.delete.forbidden (current_user_id=2, post_owner_id=1)`.

Code des tests, sortie complète et démonstrations figurent en annexe [A.4](#).

## 4.12 Jeu d'essai de la fonctionnalité la plus représentative

La fonctionnalité retenue est « **Publier un message** » : elle traverse toutes les couches (interface, contrôleur, validation, téléversement, modèle, SQL, réponse, rendu) et concentre les protections de sécurité. Le tableau [4.5](#) en présente le jeu d'essai.

**Analyse des écarts.** Les sept cas ont été exécutés le 16 juin 2026 (`curl`, application lancée). Pour chacun, le résultat **obtenu est conforme** au résultat attendu : **aucun écart**. Le cas

| Cas                 | Donnée d'entrée     | Résultat attendu                        | Obtenu             |
|---------------------|---------------------|-----------------------------------------|--------------------|
| Nominal             | Contenu « Bonjour » | Message créé, id retourné, HTTP 200     | Conforme (id 53)   |
| Contenu vide        | ""                  | Refus « contenu vide », pas d'insertion | Conforme           |
| Image trop lourde   | Fichier > limite    | Refus « image trop lourde »             | Conforme (max 2M)  |
| Extension interdite | Fichier .php        | Refus « type de fichier non autorisé »  | Conforme           |
| Charge XSS          | <script>...         | Stocké brut, <b>rendu échappé</b>       | Conforme (échappé) |
| Non authentifié     | Aucune session      | Redirection / 403 (AuthMiddleware)      | Conforme (302)     |
| Sans jeton CSRF     | POST sans jeton     | 403 (CsrfMiddleware)                    | Conforme (403)     |

Table 4.5 – Jeu d'essai de la fonctionnalité « Publier un message ».

le plus sensible est la charge XSS : le contenu `<script>` est stocké **tel quel** en base, mais ressort **échappé** au rendu (`&lt;script&gt;`, via `Utils::e`), donc inerte dans le navigateur — la séparation « stockage brut / sortie échappée » est ainsi vérifiée de bout en bout. Le détail des requêtes et des réponses obtenues figure en annexe A.4.

### 4.13 Préparation et documentation du déploiement

La mise en production est assurée par **Dokploy**, une plateforme (PaaS) auto-hébergée sur un VPS : c'est elle qui fournit le Traefik terminant le HTTPS (Let's Encrypt), redirigeant HTTP vers HTTPS et posant l'en-tête HSTS. L'application est décrite par une pile dédiée `docker-compose.dokploy.yml` — variante de `docker-compose.prod.yml` sans Traefik, pour ne pas entrer en conflit avec celui de Dokploy (deux Traefik se battant pour les ports 80/443 seraient une faute de conception). Le service `migrate` applique les migrations *avant* le démarrage de l'application, et PostgreSQL n'est exposé que sur le réseau interne. La pile autonome `docker-compose.prod.yml` (Traefik embarqué) reste l'alternative « VPS nu », documentée dans `DEPLOYMENT.md` ; ses *labels* HSTS sont reproduits ci-dessous.

```

1 labels:
2   - traefik.http.routers.solage.entrypoints=websecure
3   - traefik.http.routers.solage.tls.certresolver=le
4   - traefik.http.middlewares.solage-hsts.headers.stsSeconds=31536000
5   - traefik.http.middlewares.solage-hsts.headers.stsIncludeSubdomains=true
6   - traefik.http.routers.solage.middlewares=solage-hsts

```

Listing 4.19 – Routage HTTPS et HSTS en production — `docker-compose.prod.yml`

**Procédure documentée.** La procédure complète est consignée dans `DEPLOYMENT.md`, à la racine du dépôt. Elle couvre, étape par étape :

- les **pré-requis** : serveur Linux avec Docker, nom de domaine dont l'enregistrement DNS A pointe vers le serveur (indispensable au challenge TLS-ALPN-01 de Let's Encrypt), ports 80 et 443 ouverts ;
- les **variables d'environnement** (`.env`), avec le garde-fou `${VAR:?}` qui **fait échouer le démarrage** si une valeur manque ;
- les **étapes** de mise en production : `git pull` puis `docker compose -f docker-compose.prod.yml up -d --build` ; l'ordonnancement (Postgres sain → `migrate` → application) est garanti par la pile ;
- la **vérification** (*smoke test*) : `docker compose ps` et deux `curl` contrôlant la redirection 308 (HTTP → HTTPS) et l'en-tête HSTS ;
- le **retour arrière** (*rollback*) : retour à un *tag* Git et reconstruction pour le code ; côté données, des migrations **additives et idempotentes** (sans script *down*) que l'ancien code reste capable de lire ;
- la **sauvegarde / restauration** de la base par `pg_dump`, prise avant chaque déploiement.

**Scripts de déploiement.** Le déploiement est **déclaratif** : les scripts sont les fichiers `docker-compose.yml` (développement) et `docker-compose.prod.yml` (production), complétés par `bin/migrate.php` qu'exécute le service `migrate`. Il n'y a pas de script impératif `deploy.sh` : le `docker compose up` est le déploiement, ce qui le rend reproductible et versionné avec le code.

**Environnements.** Trois environnements sont définis ; ils se distinguent par la terminaison TLS, l'exposition de PostgreSQL et le mode de chargement du code (tableau 4.6).

| Aspect                 | Développement                            | Pré-prod. ( <i>staging</i> )                                | Production                           |
|------------------------|------------------------------------------|-------------------------------------------------------------|--------------------------------------|
| Fichier <i>compose</i> | <code>docker-compose.yml</code>          | <code>docker-compose.prod.yml</code><br>+ CA <i>staging</i> | <code>docker-compose.prod.yml</code> |
| TLS                    | aucun (HTTP)                             | Let's Encrypt <i>staging</i>                                | Let's Encrypt (HTTPS, HSTS)          |
| Code                   | <i>bind-mount</i> (rechargement à chaud) | image figée                                                 | image figée                          |
| PostgreSQL             | port exposé (outils locaux)              | non exposé                                                  | non exposé                           |
| Jeu de données         | <code>seed.sql</code> chargé             | aucun                                                       | aucun                                |

**Table 4.6** – Les trois environnements de SOLAGE et leurs différences.

**Limite assumée du déploiement.** Dokploy construit l'image *sur le VPS* à partir du dépôt Git, sans *registry* d'images. En contexte professionnel, je publierais l'image sur un *registry* en la taguant par l'empreinte du *commit* (SHA) : le retour arrière se ferait alors par simple repointage de *tag*, plutôt que par reconstruction depuis Git.

## 4.14 Contribution à la mise en production (DevOps)

Plusieurs briques DevOps étaient déjà en place : conteneurisation complète (`docker compose`), images reproductibles, migrations **idempotentes** rejouables sans risque, et journalisation structurée adaptée à une collecte par l'orchestrateur. Il manquait l'**automatisation** : elle est désormais assurée par un pipeline GitHub Actions d'**intégration** et de **déploiement** continus, déclenché à chaque *push*.

**Le pipeline.** Le dépôt étant hébergé sur GitHub, le serveur d'automatisation est **GitHub Actions** (hébergé, gratuit, rien à installer). Le fichier `.github/workflows/ci.yml` décrit **trois jobs** : deux contrôles en parallèle (`qualite`, `image`), puis un déploiement conditionnel (`déploiement`) :

- `qualite` — sur **PHP 8.3** (la version de production) : installe les dépendances, vérifie le style **PSR-12** (PHP\_CodeSniffer), lance l'**analyse statique** (PHPStan), puis exécute la **suite PHPUnit** (40 tests) contre une base PostgreSQL ;
- `image` — **construit l'image Docker** (le livrable), pour qu'une image qui ne se construit plus soit détectée immédiatement, et non le jour du déploiement ;
- `déploiement` — sur `main` uniquement et **seulement si les deux précédents sont verts**, appelle le *webhook* de redéploiement de Dokploy.

```

1  name: CI
2  on: [push]
3
4  jobs:
5    qualite:
6      runs-on: ubuntu-latest
7      services:
8        postgres:                # base PostgreSQL jetable (auth trust)
9          image: postgres:16-alpine
10         env:
11           POSTGRES_DB: solage
12           POSTGRES_USER: solage
13           POSTGRES_HOST_AUTH_METHOD: trust
14         ports: ["5432:5432"]
15         options: >-
16           --health-cmd "pg_isready -U solage -d solage"
17           --health-interval 5s --health-retries 10
18     steps:
19       - uses: actions/checkout@v4
20       - uses: shivammathur/setup-php@v2
21       with:
22         php-version: '8.3'        # version de production
23         extensions: mbstring, pdo_pgsql
24       - run: composer install --no-interaction --no-progress
25       - run: php vendor/bin/phpcs --report=full
26       - run: php vendor/bin/phpstan analyse --no-progress --memory-limit=512
27         M
28       - run: psql -h localhost -U solage -d solage -f solage.pg.sql
29       - run: php vendor/bin/phpunit
30     env:

```

```

30         DB_HOST: localhost
31         DB_PORT: 5432
32         DB_NAME: solage
33         DB_USER: solage
34         DB_PASSWORD: ""           # trust : ignore par Postgres
35
36     image:
37         runs-on: ubuntu-latest
38         steps:
39             - uses: actions/checkout@v4
40             - run: docker build -t solage-app .
41
42     deploiement:                   # déploiement continu, main
43         uniquement
44         needs: [qualite, image]   # seulement si qualité ET build sont
45         if: github.ref == 'refs/heads/main'
46         runs-on: ubuntu-latest
47         steps:
48             - run: curl -fsSL -X POST "${{ secrets.DOKPLOY_WEBHOOK }}" # jeton =
49                 secret GitHub

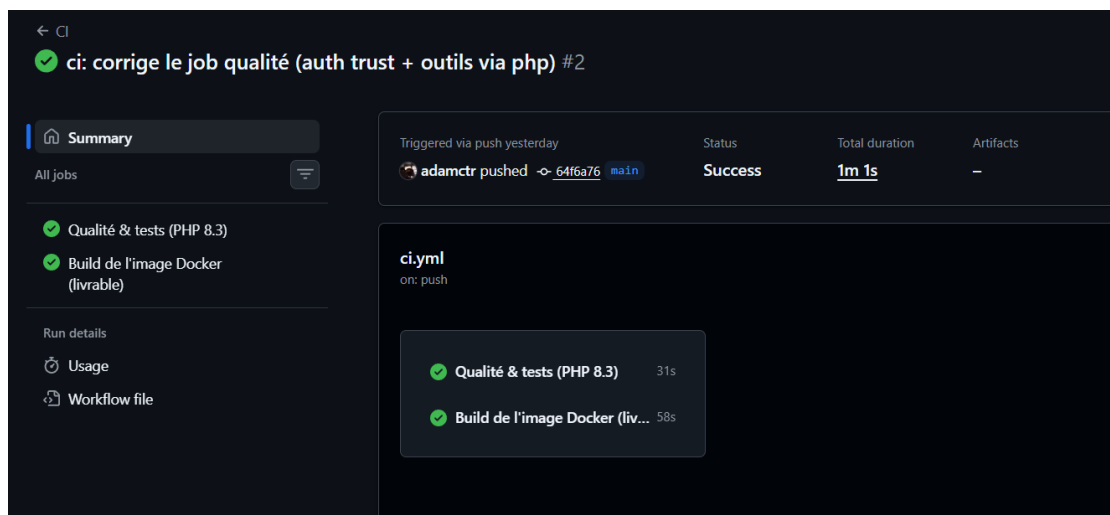
```

Listing 4.20 – Pipeline d’intégration continue — extrait de `.github/workflows/ci.yml`

**Une base de test éphémère, sans secret.** Mes tests d’intégration visent une vraie PostgreSQL (section 4.11). Le pipeline démarre donc un conteneur `postgres:16` déclaré comme *service*, attend qu’il soit sain (*healthcheck*) et y charge le schéma `solage.pg.sql`. La base est **jetable** et joignable seulement depuis le *runner* : elle utilise l’authentification `trust`, donc **aucun mot de passe n’est codé en dur** dans le pipeline. Les véritables identifiants vivent dans `.env` (hors dépôt) — même philosophie « base jetable » que les tests, portée en intégration continue.

**Analyse statique introduite avec une baseline.** PHPStan est branché au **niveau 1**. Introduit sur du code existant, il a relevé cinq alertes héritées — dont le couplage de `createUser` au `docroot` (un `include` relatif). Je les ai **gelées dans une baseline** versionnée (`phpstan-baseline.neon`) : la CI reste verte, la dette héritée est **visible et datée**, et **tout nouveau code est analysé strictement**. La baseline dégonfle à chaque correction — c’est la pratique courante pour introduire l’analyse statique sur un projet existant.

**Interprétation du rapport.** À chaque *push*, le pipeline installe PHP 8.3, vérifie PSR-12, lance PHPStan, exécute les 40 tests PHPUnit contre une PostgreSQL éphémère, et construit l’image Docker. La figure 4.11 montre un run **vert** : les deux *jobs* aboutissent — le code respecte PSR-12, l’analyse statique ne relève rien au-delà de la baseline, les 40 tests passent et le livrable se construit. Un run **rouge** bloquerait l’intégration et désignerait l’étape fautive (un test cassé par une régression, une image qui ne se construit plus) que je corrige en local avant de re-pousser. C’est cette boucle **automatique** — et non l’exécution manuelle des outils — qui garantit qu’aucune modification non vérifiée n’entre dans la branche.



**Figure 4.11** — Run d'intégration continue au vert : les contrôles de qualité et la construction du livrable aboutissent.

**De l'intégration au déploiement continu.** À chaque *push*, qualité, analyse statique, tests et construction du livrable sont rejoués automatiquement, en quelques minutes : c'est l'**intégration continue**. Sur *main* et *seulement* si ces contrôles sont verts, le *job* **déploiement** appelle le *webhook* de **Dokploy**, qui reconstruit et redéploie l'application sur le VPS : c'est le **déploiement continu**. Un *push* sur *main* qui passe la CI part donc en production sans intervention manuelle, tandis qu'un *push* sur une branche de travail exécute les contrôles *sans* déployer (condition `if: github.ref == 'refs/heads/main'`). Le jeton du *webhook* vit dans un secret GitHub, jamais en clair dans le dépôt. Distinguer ainsi les deux étapes — vérifier, puis livrer seulement ce qui est vérifié — est le cœur de la démarche DevOps.

## 4.15 Veille de sécurité

**Sources suivies.** La veille s'appuie sur l'OWASP (Top 10 et guides de test), les publications de l'ANSSI, les bulletins de sécurité de PHP et les bases de vulnérabilités (CVE).

**Le déclic IDOR.** Un article sur la fuite de données ayant touché l'URSSAF (accès aux données d'autres assurés en modifiant un identifiant dans l'URL — une faille **IDOR**) a déclenché un audit de SOLAGE. Il a révélé que les routes `/edituser/{id}`, `/api/users/delete` et `/api/posts/delete` vérifiaient l'*authentification* (`AuthMiddleware`) mais pas la *propriété* de la ressource : un utilisateur authentifié pouvait modifier ou supprimer le contenu d'autrui en changeant l'identifiant. Le correctif (contrôle « propriétaire ou administrateur » + 403 + journalisation) est décrit en section 4.7.

**Journal de veille.** Chaque élément de veille marquant est consigné avec sa source, ce qu'il m'a appris et son impact concret sur SOLAGE (tableau 4.7). La veille n'est pas un exercice séparé du code : elle déclenche des audits et des correctifs.

| Date      | Source                                                                           | Apprentissage                                                                                                         | Impact sur Solage                                                                                                                                                                  |
|-----------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mai 2026  | Article sur la fuite de données à l'URSSAF (faible IDOR)                         | Une ressource accessible par identifiant sans contrôle de propriété = IDOR (OWASP A01, <i>Broken Access Control</i> ) | Audit des routes <code>/edituser/{id}</code> , <code>/api/users/delete</code> , <code>/api/posts/delete</code> : contrôle « propriétaire ou administrateur », 403 + journalisation |
| Mai 2026  | OWASP Top 10 (2021), OWASP Secure Headers, recommandations ANSSI                 | Les en-têtes HTTP de sécurité (CSP, HSTS, <code>nosniff</code> ) forment une défense en profondeur côté navigateur    | Ajout des en-têtes au <i>bootstrap</i> ; HSTS posé par Traefik en production                                                                                                       |
| Juin 2026 | OWASP — <i>CSRF Prevention Cheat Sheet</i> ( <i>Synchronizer Token Pattern</i> ) | L'attribut <code>SameSite</code> seul ne suffit pas; un jeton applicatif protège indépendamment du navigateur         | Jetons CSRF armés par le routeur sur tout <code>POST</code>                                                                                                                        |

Table 4.7 — Journal de veille de sécurité et son impact sur SOLAGE.

## Chapitre 5

# Bilan, difficultés et perspectives

### 5.1 Satisfactions

Le projet a permis de construire et de *posséder* une application web multicouche complète : un socle « framework » écrit à la main (routeur, middlewares, logger PSR-3, utilitaires), une sécurité pensée couche par couche (XSS, CSRF, injection SQL, IDOR), une base PostgreSQL conçue proprement avec migrations idempotentes, et une chaîne Docker reproductible du développement à la production. La migration de MariaDB vers PostgreSQL et l’audit d’architecture ont particulièrement consolidé la compréhension du fonctionnement interne de l’application.

### 5.2 Difficultés rencontrées et résolues

Ces difficultés démontrent la **démarche structurée de résolution de problème** (compétence transversale T2) : chacune suit le schéma symptôme → diagnostic → correctif.

**Headers already sent à la connexion.** La réponse JSON était émise *avant session\_start()*, empêchant la pose du cookie PHPSESSID. *Correctif* : démarrer la session dans le *bootstrap* (`public/index.php`), avant toute sortie possible.

**Requête N+1 sur le fil.** L’affichage des auteurs déclenchait une requête par message. *Correctif* : préchargement en une requête IN (section 4.8), avec gestion du cas IN () vide, interdit en PostgreSQL.

### 5.3 Perspectives

Les axes d’amélioration, déjà identifiés dans le dossier, sont :

- le **durcissement de la validation serveur** (format de l’email, robustesse du mot de passe) et l’**anti-fixation de session** (`session_regenerate_id`);
- la **validation du téléversement par type MIME réel** (`finfo / getimagesize`) en complément de l’extension;
- une passe d’**accessibilité** (RGAA) plus poussée;
- la **maintenabilité du front-end** : séparation et factorisation des feuilles de style et des scripts, **chargement conditionnel du JavaScript** selon le gabarit (*layout*) plutôt qu’un script global, et mise en place d’un *lint* JavaScript (ESLint).



# Annexe A

## Annexes

Les annexes (40 pages maximum) rassemblent les éléments de la fonctionnalité la plus représentative : maquettes, captures et code, code des composants métier et d'accès aux données, code des autres composants, et jeux de tests complets.

### A.1 Gestion de projet : suivi des tâches

Le fichier `SUIVI.md`, consolidé depuis l'historique Git, recense les tâches du projet par phase (tâche, phase, date). Le projet s'est déroulé en deux temps : une construction initiale (2024), puis une reprise dédiée à la sécurisation et à l'industrialisation (2026).

#### **Phase 1 — Construction initiale (sept.–nov. 2024).**

- page d'accueil et fil d'actualité (front) ;
- « j'aime » (likes) et appels AJAX ;
- routeur maison et gestion des utilisateurs ;
- module de réponses imbriquées, navigation et routes ;
- minification des assets, comptage récursif des réponses.

#### **Phase 2 — Reprise : sécurisation & industrialisation (mai–juin 2026).**

- conteneurisation (Docker) et migration de la base vers PostgreSQL ;
- gestion des secrets (`.env`) et logger PSR-3 ;
- `AdminMiddleware` et protection des routes d'administration ;
- échappement anti-XSS et contrôle d'accès anti-IDOR sur la suppression ;
- refonte MVC (correction du N+1, découplage du validateur, unification JSON) ;
- protection CSRF, en-têtes de sécurité, injection de dépendance de session ;
- jeu d'essai (`seed.sql`), `PHP_CodeSniffer` (PSR-12), refonte de la feuille de style.

### A.2 Script de création de la base de données

Script complet `solage.pg.sql`, chargé à l'initialisation du conteneur PostgreSQL.

```
1 BEGIN ;  
2
```

```

3 CREATE TABLE roles (
4     id SERIAL PRIMARY KEY,
5     name VARCHAR(255) NOT NULL
6 );
7 INSERT INTO roles (id, name) VALUES (1, 'Admin'), (2, 'Utilisateur'), (3, 'Modé
    rateur');
8 SELECT setval(pg_get_serial_sequence('roles', 'id'),
9             (SELECT COALESCE(MAX(id), 1) FROM roles));
10
11 CREATE TABLE users (
12     id SERIAL PRIMARY KEY,
13     name VARCHAR(255) NOT NULL,
14     firstname VARCHAR(255) NULL,
15     email VARCHAR(255) NOT NULL UNIQUE,
16     password VARCHAR(255) NOT NULL,
17     role INT NULL REFERENCES roles(id) ON DELETE SET NULL,
18     image VARCHAR(255) NULL
19 );
20 CREATE INDEX users_role_idx ON users(role);
21
22 CREATE TABLE posts (
23     id SERIAL PRIMARY KEY,
24     user_id INT NULL REFERENCES users(id) ON DELETE CASCADE,
25     date TIMESTAMP NOT NULL,
26     likes INT DEFAULT 0,
27     content TEXT NULL,
28     reply_to INT NULL,          -- message auquel on répond (NULL =
        post racine)
29     reply_to_parent INT NULL,   -- racine du fil de discussion
30     image TEXT NULL
31 );
32 CREATE INDEX posts_user_idx ON posts(user_id);
33
34 CREATE TABLE likes (
35     id SERIAL PRIMARY KEY,
36     user_id INT NULL REFERENCES users(id) ON DELETE CASCADE,
37     post INT NULL REFERENCES posts(id) ON DELETE CASCADE,
38     created_at TIMESTAMP NOT NULL
39 );
40 CREATE INDEX likes_user_idx ON likes(user_id);
41 CREATE INDEX likes_post_idx ON likes(post);
42
43 COMMIT;

```

### A.3 Code d'un autre composant : le routeur

Routeur complet `src/Router.php` : correspondance d'URL, exécution des middlewares, armement du CSRF sur les requêtes POST, et instanciation du contrôleur.

```

1 class Router {

```

```

2     protected $routes = [];
3
4     public function addRoute($method, $path, $target, $middleware = null) {
5         $this->routes[] = ['method' => $method, 'path' => $path,
6                             'target' => $target, 'middleware' => $middleware
7                             ];
8     }
9
10    public function match() {
11        $requestUri      = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
12        $requestMethod    = $_SERVER['REQUEST_METHOD'];
13
14        foreach ($this->routes as $route) {
15            $pathRegex = preg_replace('/\{([a-zA-Z0-9_]+\})/', '([a-zA-Z0-9_
16                ]+)', $route['path']);
17            $pathRegex = '#^' . $pathRegex . '$#';
18
19            if ($route['method'] === $requestMethod && preg_match($pathRegex
20                , $requestUri, $matches)) {
21                if ($route['middleware']) {
22                    (new $route['middleware']())->handle();
23                }
24                if ($route['method'] === 'POST') {
25                    (new CsrfMiddleware())->handle();
26                }
27                $routeArray = explode('#', $route['target']);
28                if (count($routeArray) < 2) {
29                    throw new Exception("Le 'target' doit etre au format '
30                        Controller#Method'");
31                }
32                [$controller, $functionController] = $routeArray;
33                if (isset($matches[1])) {
34                    $instance = new $controller($matches[1]);
35                    call_user_func_array([$instance, $functionController],
36                        $matches);
37                    return;
38                }
39                (new $controller())->$functionController();
40                return;
41            }
42        }
43        http_response_code(404);
44        page404View::show();
45    }
46 }

```

## A.4 Jeux de tests complets

Les tests sont écrits avec PHPUnit 11 et organisés en trois niveaux (unitaire, intégration, sécurité). Ils se trouvent sous `tests/` et s'exécutent en une commande (`php vendor/bin/phpunit`).

|                        |                                        |      |
|------------------------|----------------------------------------|------|
| tests/                 |                                        |      |
| bootstrap.php          | autoloader maison + Database           |      |
| UtilsTest.php          | echappement XSS, AJAX, reponse JSON    | (11) |
| CsrfHelperTest.php     | jeton CSRF : generation, verification  | (8)  |
| SessionManagerTest.php | login / isAdmin au mock (DI, zero BDD) | (3)  |
| Integration/           |                                        |      |
| DatabaseTestCase.php   | transaction + rollback par test        |      |
| UserModelTest.php      | CRUD, hachage, UserValidator           | (10) |
| PostModelTest.php      | creation, lecture, suppression         | (6)  |
| SqlInjectionTest.php   | charge d'injection inerte              | (2)  |

Listing A.1 – Arborescence des tests

**Exemple — test d'intégration sécurité (injection SQL)**

```

1 final class SqlInjectionTest extends DatabaseTestCase
2 {
3     // setUp() insere 1 utilisateur + 1 post au contenu unique (MARQUEUR),
4     // le tout dans la transaction annulee au teardown.
5
6     public function une_charge_d_injection_reste_inerte(): void
7     {
8         // "%' OR '1'='1%" est cherche litteralement : 0 resultat, sans
9         // exception.
10        $r = (new SearchModel())->searchPosts("' OR '1'='1");
11        $this->assertCount(0, $r);
12    }
13
14    public function une_recherche_normale_fonctionne(): void
15    {
16        $r = (new SearchModel())->searchPosts(self::MARQUEUR);
17        $this->assertCount(1, $r);
18    }
19 }

```

Listing A.2 – tests/Integration/SqlInjectionTest.php (extrait)

**Sortie complète d'exécution**

```

Csrf Helper
OK GetToken genere 64 caracteres hexadecimaux
OK GetToken est stable dans une meme session
OK VerifyToken accepte le bon token
OK VerifyToken rejette un mauvais token
OK VerifyToken rejette une chaine vide
OK VerifyToken rejette null
OK VerifyToken rejette quand aucun token en session
OK Field rend un input cache avec le token

Post Model
OK CreatePost insere et renvoie un id
OK GetPostById lit le post cree
OK GetPostById retourne null si inconnu

```

```

OK Delete supprime le post
OK Delete retourne false si post inexistant
OK GetAllPostsByUserId ne retourne que les posts de l auteur
Session Manager
OK Login peuple la session depuis le modele
OK IsAdmin vrai pour le role admin
OK IsAdmin faux pour un role non admin
Sql Injection
OK Une charge d injection reste inerte
OK Une recherche normale fonctionne
User Model
OK GetUserByEmail retourne l utilisateur existant
OK GetUserByEmail retourne null si inconnu
OK Le mot de passe est stocke hashe
OK CreateUser stocke un mot de passe hashe
OK Login valide avec les bons identifiants
OK Login refuse un mauvais mot de passe
OK Login refuse un email inexistant
OK Login refuse des champs vides
OK Register refuse un email deja utilise
OK Register accepte un email libre
Utils
OK E neutralise une balise script
OK E encode les chevrons
OK E encode les guillemets doubles
OK E encode l apostrophe en apos
OK E encode l esperluette
OK E transforme null en chaine vide
OK IsAjax vrai avec l entete xmlhttprequest
OK IsAjax faux sans l entete
OK SendResponse serialise success et message
OK SendResponse inclut data quand non vide
OK SendResponse omet data quand falsy

OK (40 tests, 62 assertions)

```

Listing A.3 – php vendor/bin/phpunit -testdox

**Démonstrations de sécurité fonctionnelles.** Rejouées en curl, application lancée; preuve = statut HTTP + ligne de journal (le *logger* écrit les avertissements sur stderr, collectés par Docker).

```

$ curl -i -X POST http://localhost/login
HTTP/1.1 403 Forbidden
403 - Token CSRF refuse.
    journal: {"level":"warning","msg":"csrf.token.rejected","context":{"uri":
              "":"/login"}}

$ curl -i -X POST ../api/posts/delete    (session Bob, postId d'Alice = 1)
HTTP/1.1 403 Forbidden
{"success":false,"message":"Vous n'avez pas la permission de supprimer ce

```

```

    post."}
  journal: {"level":"warning","msg":"post.delete.forbidden",
           "context":{"current_user_id":2,"post_owner_id":1,"post_id":1}}

```

**Listing A.4** – CSRF (POST sans jeton) et IDOR (Bob supprime un message d’Alice)

### Jeu d’essai « Publier un message » — données obtenues (16 juin 2026)

|                   |                                                                |
|-------------------|----------------------------------------------------------------|
| Nominal           | -> success=true, id=53, HTTP 200                               |
| Contenu vide      | -> success=false, "Donnees invalides ou contenu vide"          |
| Image trop lourde | -> success=false, "Image trop lourde (max 2M)."                |
| Extension .php    | -> success=false, "Type de fichier non autorise."              |
| Charge XSS        | -> stocke brut "<script>...", rendu "&lt;script&gt;" (echappe) |
| Non authentifie   | -> HTTP 302 -> /login (AuthMiddleware)                         |
| Sans jeton CSRF   | -> HTTP 403 (CsrfMiddleware)                                   |

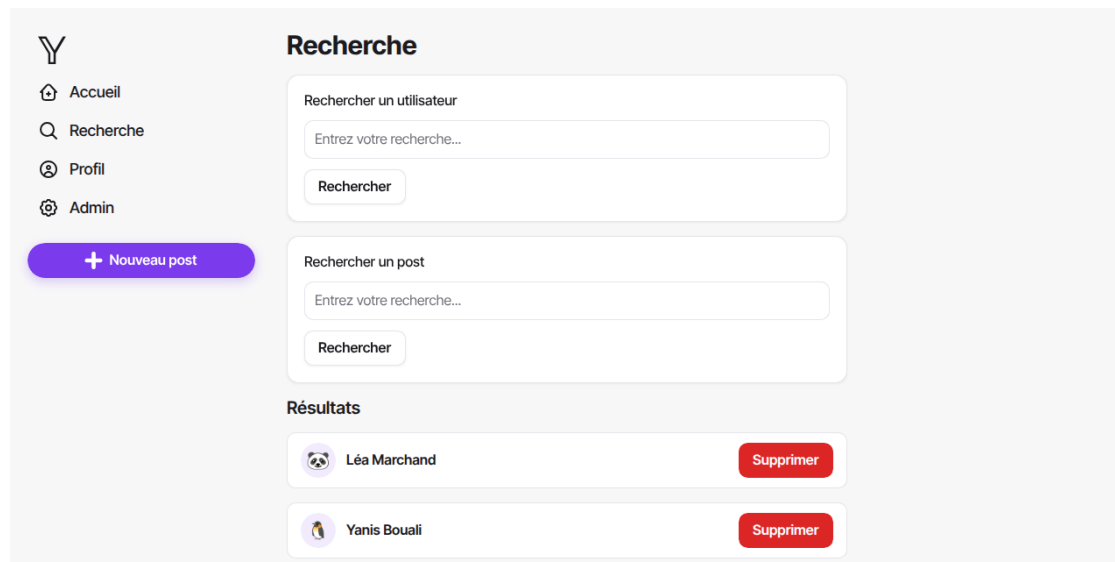
**Listing A.5** – Réponses obtenues pour les sept cas

## A.5 Maquettes et captures d’écran

Maquettes des écrans de SOLAGE réalisées sous Penpot (versions bureau et mobile), complétant les trois écrans structurants présentés dans le corps du dossier (section 4.3).

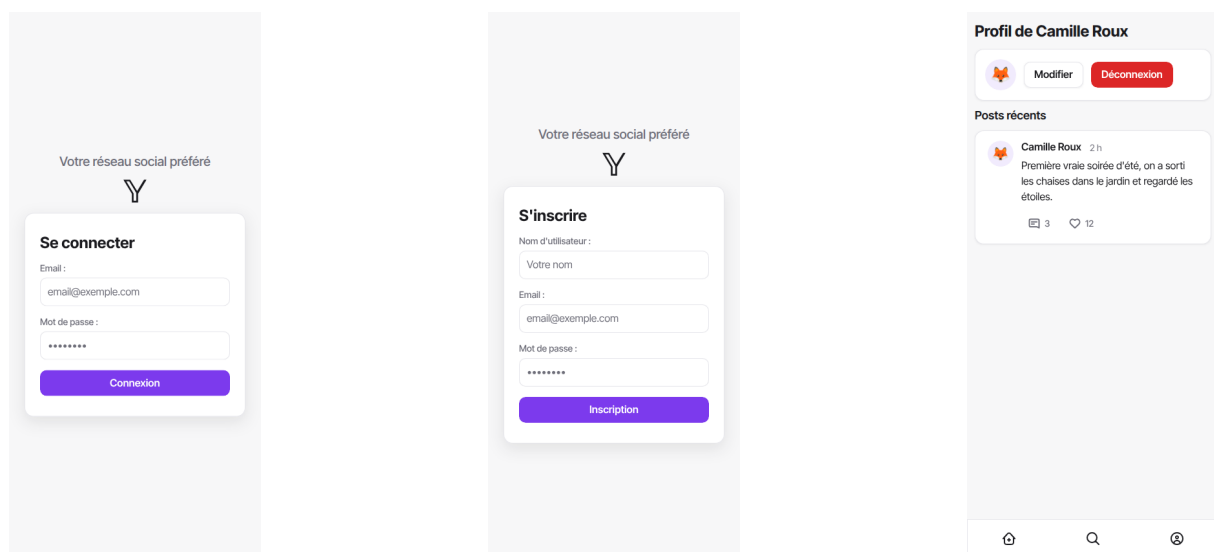
### A.5.1 Écrans bureau

**Figure A.1** – Inscription (bureau) : même gabarit que la connexion, étendu aux champs du compte.



**Figure A.2** — Vue bureau avec barre latérale de navigation persistante (accueil, recherche, profil, administration) : recherche d'utilisateurs et de messages, et résultats assortis de l'action de modération « Supprimer ».

### A.5.2 Écrans mobiles



**Figure A.3** — Mobile : connexion, inscription et profil utilisateur.



**Figure A.4** – Mobile : recherche et administration. La barre de navigation inférieure reste constante.

**Code source complet.** Le code source complet de SOLAGE (contrôleurs, modèles, vues, middlewares, configuration Docker) est disponible dans le dépôt Git du projet. Les extraits présentés dans ce dossier sont les plus significatifs au regard des compétences évaluées.